



Decomposition of Supply Networks into Subnetworks for Reducing Computational Complexity in Mixed-Integer Optimization Problems

Bachelor's Thesis

As part of the examination:

Bachelor of Science (B. Sc.)

in Computer Science

at the
Baden-Württemberg Cooperative State University Karlsruhe

by

Max Katzenberger

Submission Date	August 31, 2026
Processing Period	June 8, 2026 – August 31, 2026
Matriculation Number, Course	4008573, TINF23B2
Training Company	SAP SE Dietmar-Hopp-Allee 16 69190 Walldorf, Deutschland
Department	SCM Optimizer
Supervisor at Training Company	Christoph Eckert
Supervisor at University	Prof. Dr. Heinrich Braun

Declaration of Originality

I hereby declare that I have independently written my Bachelor's Thesis with the topic:

> *Decomposition of Supply Networks into Subnetworks for Reducing Computational Complexity in Mixed-Integer Optimization Problems*

in accordance with § 5 of the "Studien- und Prüfungsordnung DHBW Technik" dated September 29, 2017 and have not used any sources or aids other than those specified. The work has not been submitted to any other examining authority before and has not been published.

Furthermore, I confirm that the submitted electronic version corresponds to the printed version.

Karlsruhe, August 31, 2026

_____

Katzenberger, Max

Abstract

- English -

Large-scale supply chain optimization problems often exceed the computational capabilities of monolithic mixed-integer linear programming solvers, resulting in long solve times or failure to find feasible solutions within practical time constraints. This thesis develops and evaluates a stage-based decomposition approach that partitions supply chain networks along their natural boundaries into procurement, production and distribution subnetworks. The subnetworks are solved sequentially through a two-phase procedure: a bottom-up phase propagates demand requirements from customers toward suppliers, followed by a top-down phase that allocates actual supplies based on upstream capacity. Link nodes at partition boundaries enable information exchange between subnetworks, while aggregation heuristics propagate cost, lead time and capacity information to guide local optimization decisions.

Evaluation on 13 real customer scenarios from SAP Integrated Business Planning demonstrates the effectiveness of the approach. Across the 10 scenarios for which both approaches produced feasible solutions, decomposition achieved an average relative objective-value improvement of 23% compared to monolithic solving under identical solver-time budgets. For large-scale scenarios with high monolithic optimality gaps, improvements reach up to 92%. The approach proves particularly effective for supply chains with clear stage separation, achieving near-optimal solutions. Decomposition also enables solving scenarios that exceed monolithic solver capabilities entirely, producing feasible solutions where monolithic approaches fail to find any solution within the given time budget.

Abstract

- Deutsch -

Große Optimierungsprobleme in der Supply-Chain-Planung übersteigen häufig die Rechenkapazitäten monolithischer MILP-Solver, was zu langen Lösungszeiten führt oder dazu, dass innerhalb praktischer Zeitvorgaben keine zulässige Lösung gefunden wird. Diese Arbeit entwickelt und evaluiert einen stufenbasierten Dekompositionsansatz, der Supply-Chain-Netzwerke entlang ihrer natürlichen Grenzen in Beschaffungs-, Produktions- und Distributionssubnetzwerke aufteilt. Die Subnetzwerke werden sequenziell durch ein zweiphasiges Verfahren gelöst: Eine Bottom-up-Phase propagiert Bedarfsanforderungen von den Kunden zu den Lieferanten, gefolgt von einer Top-down-Phase, die tatsächliche Liefermengen basierend auf der vorgelagerten Kapazität zuweist. Verbindungsknoten an den Partitionsgrenzen ermöglichen den Informationsaustausch zwischen Subnetzwerken, während Aggregationsheuristiken Kosten-, Lieferzeit- und Kapazitätsinformationen weitergeben, um lokale Optimierungsentscheidungen zu verbessern.

Die Evaluation anhand von 13 realen Kundenszenarien aus SAP Integrated Business Planning zeigt die Wirksamkeit des Ansatzes. Für die 10 Szenarien, bei denen beide Ansätze zulässige Lösungen lieferten, erzielte die Dekomposition unter identischen Solver-Zeitbudgets eine durchschnittliche relative Kostenverbesserung von 23% gegenüber dem monolithischen Ansatz. Bei großen Szenarien mit hohem monolithischem Gap erreichen die Verbesserungen bis zu 92%. Der Ansatz erweist sich als besonders effektiv für Supply Chains mit klarer Stufentrennung und erzielt nahezu optimale Lösungen. Die Dekomposition ermöglicht zudem das Lösen von Szenarien, bei denen monolithische Solver an ihre Grenzen stoßen, und liefert zulässige Lösungen, wo monolithische Ansätze innerhalb des gegebenen Zeitbudgets keine Lösung finden.

Table of Contents

List of Figures ..... **VI**

List of Tables ..... **VII**

1 Introduction **1**
1.1 Motivation and Problem 1
1.2 Approach 1
1.3 Goal and Research Questions 2

2 Theoretical Foundations **3**
2.1 Mixed-Integer Programming 3
2.2 SAP Supply Chain Planning 5

3 Related Work **12**

4 Methodology **14**
4.1 Research Approach 14
4.2 Evaluation Design 15

5 Decomposition Framework Design **19**
5.1 Graph-based Problem Modeling 19
5.2 Decomposition 20
5.3 Two-Phase Solve Approach 26
5.4 Link Node Heuristics 30
5.5 Inter-Subnetwork Constraint Handling 33
5.6 Solution Combination 34
5.7 Runtime Allocation 36

6 Implementation **38**
6.1 System Architecture 38
6.2 Data Handling and Technologies 40
6.3 Solution Verification 42

7 Evaluation **44**

8 Discussion **54**

9 Future Work **58**

10 Conclusion **63**

Bibliography ..... **a**

List of Appendices ..... **A**

List of Abbreviations

***BOM* – Bill of Materials** 6, 10, 30, 31, 32, 33

***DAG* – Directed Acyclic Graph** 22, B

***DSR* – Design Science Research** 14

***FCFS* – First-Come, First-Served** 34, 60, 61

***IBP* – Integrated Business Planning** 1, 3, 5, 6, 9, 15, 16, 39, 64

***LP* – Linear Programming** 3, 4, 10

***MILP* – Mixed-Integer Linear Programming** 1, 2, 3, 4, 5, 9, 10, 12, 13, 14, 15, 19, 30, 36, 42, 60, 63, D

***PoC* – Proof-of-Concept** 38

***SCC* – Strongly Connected Component** 22

List of Figures

Figure 1 Example of a supply chain graph with different node types. Source: Internal SupplyChainViewer. 8
Figure 2 Decomposition of a supply chain graph into three subnetworks with link nodes and virtual procurements. Source: Internal SupplyChainViewer. 21
Figure 3 Example stage classification of nodes. Source: Internal SupplyChainViewer. 23
Figure 4 Example meta graph M for three subnetworks 28
Figure 5 System architecture of the decomposition framework showing the orchestration layer and solver service interaction 38
Figure 6 Network visualization showing regular structure and clear stage separation 48

List of Tables

Table 1 Benchmark scenario characteristics 17
Table 2 Scenario structural characteristics 18
Table 3 Supply chain stage classification 22
Table 4 Virtual procurement parameters 30
Table 5 Runtime budgets for benchmark scenarios 44
Table 6 Objective function values and optimality gaps for monolithic and decomposition approaches 45
Table 7 Subproblem sizes for scenario S9 47
Table 8 Model comparison of S6 production subnetwork (bottom-up vs. top-down) 48
Table 9 Solver behavior comparison for S6 production subnetwork 49
Table 10 Top-down vs. bottom-up performance across scenarios 49
Table 11 LP relaxation solver phases for S8 top-down production subnetwork 50
Table 12 Complexity distribution across subnetworks for scenarios with worse decomposition results 51
Table 13 Demand fulfillment rates for scenarios with worse decomposition results . 52

1 Introduction

1.1 Motivation and Problem

Modern supply chain networks have grown increasingly complex, spanning multiple continents and involving thousands of suppliers, production facilities, warehouses and customer demands [1]. Planning these networks requires determining optimal production quantities, transportation quantities and inventory levels across the entire network and planning horizon. These decisions are typically formulated as *Mixed-Integer Linear Programming (MILP)* problems, which simultaneously optimize continuous variables and discrete variables [2].

However, the computational complexity of *MILP* problems poses significant challenges for enterprise-scale supply chains. *MILP* is NP-hard in general [3], meaning that solving time can grow exponentially with problem size. Real-world scenarios can involve optimization problems with millions of decision variables. Solving such large-scale problems using traditional monolithic approaches often leads to excessive computation times or exceeds available memory. This scalability bottleneck represents a critical limitation for supply chain planning systems such as SAP *Integrated Business Planning (IBP)* [4]. The challenge lies in finding methods that can handle the scale of modern supply networks while maintaining solution quality and improving computational speed.

1.2 Approach

A promising approach to address this challenge is decomposition. The core idea is to partition a large problem into smaller subproblems whose solutions combine into a global solution [5]. Graph-based decomposition strategies are particularly well-suited for supply chain networks, as these networks naturally exhibit hierarchical structures that lend themselves to partitioning along stage boundaries. The material flow in most supply chains follows three fundamental phases: sourcing raw materials and components from suppliers (Procurement), transforming inputs into finished goods (Production) and delivering products to customers (Distribution). These phases represent natural boundaries for decomposition because material flows sequentially from one phase to the next, creating clear interfaces between them. This thesis therefore proposes reducing computational complexity by decomposing supply chains along these stages,

partitioning the underlying *MILP* accordingly, solving each subproblem separately and combining the partial solutions into a feasible global solution.

1.3 Goal and Research Questions

The goal of this thesis is to develop and evaluate a stage-based decomposition strategy for supply chain network optimization problems. This involves designing a partitioning scheme that respects the network's hierarchical structure, developing a coordination mechanism that propagates information between dependent subproblems and ensuring that the combined result remains globally feasible.

This thesis addresses the following research questions:

1. **RQ1: How can supply chain networks be decomposed into subnetworks while preserving solution feasibility?**
   This question investigates graph-based decomposition strategies that partition the network while maintaining the structural and logical dependencies necessary for a valid combined solution.
2. **RQ2: How does decomposition affect solution quality and computational performance compared to monolithic solving?**
   This question examines whether decomposition can provide advantages in solution quality under the same conditions as the monolithic approach.
3. **RQ3: Can decomposition enable solving previously intractable problems?**
   This question evaluates whether the proposed approach can successfully solve large-scale scenarios that would otherwise exceed the computational resource limits of runtime or memory.

2 Theoretical Foundations

This chapter establishes the theoretical foundations underlying the decomposition approach developed in this thesis. Beginning with *MILP*, this chapter introduces its formal definition, discusses computational complexity, and motivates the need for decomposition approaches. The chapter then examines how SAP *IBP* Supply Planning models supply chains as directed graphs and transforms them into mathematical optimization problems.

2.1 Mixed-Integer Programming

MILP is a fundamental optimization paradigm that extends *Linear Programming (LP)* by requiring some variables to take integer values. A Mixed-Integer Program is an optimization problem of the following form [2]:

$$\begin{aligned} \min \quad & c^T x \\ \text{subject to} \quad & Ax \leq b \\ & x_i \in \mathbb{Z} \quad \forall i \in I \\ & x_i \in \mathbb{R} \quad \forall i \notin I \\ & x \geq 0 \end{aligned} \tag{1.}$$

where $c \in \mathbb{R}^n$ is the objective coefficient vector, $A \in \mathbb{R}^{m \times n}$ is the constraint matrix, $b \in \mathbb{R}^m$ is the right-hand side vector and $I \subseteq \{1, ..., n\}$ is the index set of variables constrained to integer values.

Several important special cases arise from this general formulation:

- **Pure Integer Program**: When $I = \{1, ..., n\}$, all variables must be integer-valued.
- **Binary Program**: When $x_i \in \{0, 1\}$ for all $i \in I$, variables are restricted to binary values.
- **Linear Program**: When $I = \emptyset$, no integrality constraints exist, yielding a standard LP.

The computational complexity of solving a *MILP* places it in the class of NP-hard problems [3]. The fundamental difficulty arises from the integrality constraints. While the feasible region of a linear program is a convex polyhedron [6], requiring even one variable to take integer values transforms the feasible region into a discrete, non-convex set, making the optimization problem NP-hard in general [3].

The source of this hardness lies in the combinatorial explosion of possible integer assignments. If each integer variable is restricted to a finite domain D_i, the search space contains $\prod_i |D_i|$ possible assignments before accounting for the linear constraints.

This exponential complexity has direct implications for solution algorithms. Unlike *LP*, where the interior point method finds optimal solutions in polynomial time, no known algorithm solves general *MILPs* in polynomial time [6].

Every *MILP* has an associated LP relaxation. The LP relaxation is obtained by replacing the variables with integer domains with continuous domains, allowing all integer-constrained variables to take fractional values [2]:

$$z_{\text{LP}} = \min\{c^T x : Ax \leq b, x \in \mathbb{R}^n, x \geq 0\} \quad 2.$$

This relaxation possesses several important properties. For minimization problems, the LP relaxation provides a lower bound on the optimal *MILP* objective value: $z_{\text{LP}} \leq z_{\text{MILP}}$. This relationship holds because the feasible region of the LP relaxation contains all integer-feasible solutions [2]. The quality of the relaxation is measured by the integrality gap:

$$\text{gap} = \frac{z_{\text{MILP}} - z_{\text{LP}}}{|z_{\text{MILP}}|} \quad 3.$$

A small integrality gap indicates that the LP relaxation closely approximates the *MILP*, making it a "tight" relaxation. While solving general *MILPs* is NP-hard, linear programs can be solved in polynomial time [6], making LP relaxation relevant for bounding and approximation purposes.

The exponential worst-case complexity motivates the use of decomposition approaches. The general idea is to exploit the structure of a large optimization problem by dividing it into smaller subproblems, thereby reducing the complexity of the search space. Consider a *MILP* with n binary variables, the number of possible assignments can grow exponentially as 2^n. If the problem can be decomposed into k smaller subproblems of size $\frac{n}{k}$, the corresponding search space is reduced to approximately $k \cdot 2^{\frac{n}{k}}$, illustrating the potential computational benefits of decomposition.

This scaling behavior makes decomposition particularly attractive for large-scale optimization. However, decomposition approaches introduce additional challenges. When subproblems are coupled through shared variables or constraints, solving them independently may no longer preserve global optimality. While exact decomposition methods can maintain optimality guarantees, they often require additional coordination and

iterative procedures [7], [8]. In practice, large-scale *MILPs* are frequently not solved to proven optimality due to computational limits, but rather until a satisfactory solution is obtained within a given time budget. Therefore, approximate decomposition approaches may trade optimality guarantees for computational efficiency, aiming to obtain high-quality solutions within reasonable time limits [9].

Modern *MILP* solvers use branch-and-bound algorithms to systematically explore the solution space [2]. The algorithm works by recursively partitioning the problem. At each node in the search tree, the LP relaxation is solved to obtain a lower bound. If the relaxation's solution satisfies all integrality constraints, an integer-feasible solution has been found. Otherwise, the algorithm selects a fractional variable and creates two child nodes by adding constraints that fix the variable to either its floor or ceiling value. Branches can be pruned when their LP relaxation bound exceeds the best known integer solution, eliminating large portions of the search space without explicit enumeration. The efficiency of branch-and-bound thus depends heavily on how quickly each LP relaxation can be solved, as thousands or millions of LP solves may be required during a single *MILP* optimization.

Modern LP solvers employ two main algorithmic approaches: simplex methods and interior-point (Barrier) methods [10]. The feasible region of an LP forms a convex polyhedron and the optimal solution always lies at a vertex of this polyhedron. Simplex methods traverse the vertices of the feasible polyhedron along its edges, maintaining a basic feasible solution at each step and converging directly to an optimal vertex. The Barrier method instead traverses the interior of the feasible region, converging to a point near the optimal vertex but not exactly at it. Since branch-and-bound requires a vertex solution as a starting point for subsequent child nodes, Barrier solutions must be converted through a Crossover phase that moves the interior-point solution to the nearby optimal vertex [11]. Modern solvers like Gurobi often run simplex and Barrier concurrently, using whichever finishes first. However, if the LP formulation exhibits numerical difficulties such as degeneracy, the Crossover phase can become extremely slow or fail entirely, causing severe performance degradation in the overall *MILP* solve.

2.2 SAP Supply Chain Planning

SAP *IBP* Supply Planning provides an enterprise platform for supply chain optimization that transforms real-world supply networks into mathematical models [4]. This section

introduces the conceptual foundations of how SAP Supply Planning represents supply chains and formulates optimization problems, establishing the context for the decomposition approaches developed in this thesis.

2.2.1 Supply Chains as Graphs

Modern supply chains consist of connected products, operations and customers that can be naturally represented as directed graphs [1]. SAP *IBP* Supply Planning leverages this graph-theoretic perspective to model supply networks, where vertices represent operational entities and edges capture material flows and dependencies.

The foundation of Supply Planning is a relational data model organized into several categories of input tables [4]. Example categories are given by:[1]

- **Location-Product Tables** define which products exist at which locations in the supply network, establishing the basic inventory points.
- **Customer-Product Tables** represent a product at a customer, modeling possible demand points where customer orders may occur.
- **Production Tables** define manufacturing operations through production rules that encode *Bill of Materials (BOM)* relationships, specifying which input materials and in what quantities are required to produce each output product.
- **Transport Tables** describe logistics lanes between locations, capturing lead times, capacities and transportation costs for moving materials through the network.
- **Customer-Transport Tables** define delivery lanes from locations to customers, specifying how finished products reach demand points.
- **Procurement Tables** model external supply sources, defining available quantities, costs and constraints for purchasing materials from suppliers.
- **Resource Tables** capture capacity constraints for production equipment, storage facilities and transportation vehicles that limit operational throughput.

Each table contains time-indexed key figures that specify period-by-period values for quantities, costs and capacity limits across the planning horizon.

From these relational tables, a directed graph $G = (V, E)$ can be constructed to capture the supply chain topology. This graph abstraction makes explicit the relationships between entities that are implicit in the relational tables, enabling structured parti-

[1] The tables vary based on interface, version and release, but can generally be categorized by those examples.

tioning for decomposition approaches. By dividing the graph into subgraphs, it can be directly determined which portions of the input data belong to each subproblem. The vertex set V comprises the node types that represent the distinct operational entities defined by the tables above.

The edge set E captures the material flow and dependencies of resources on operations. Figure 1 illustrates an example of such a supply chain graph with material flow from top to bottom.

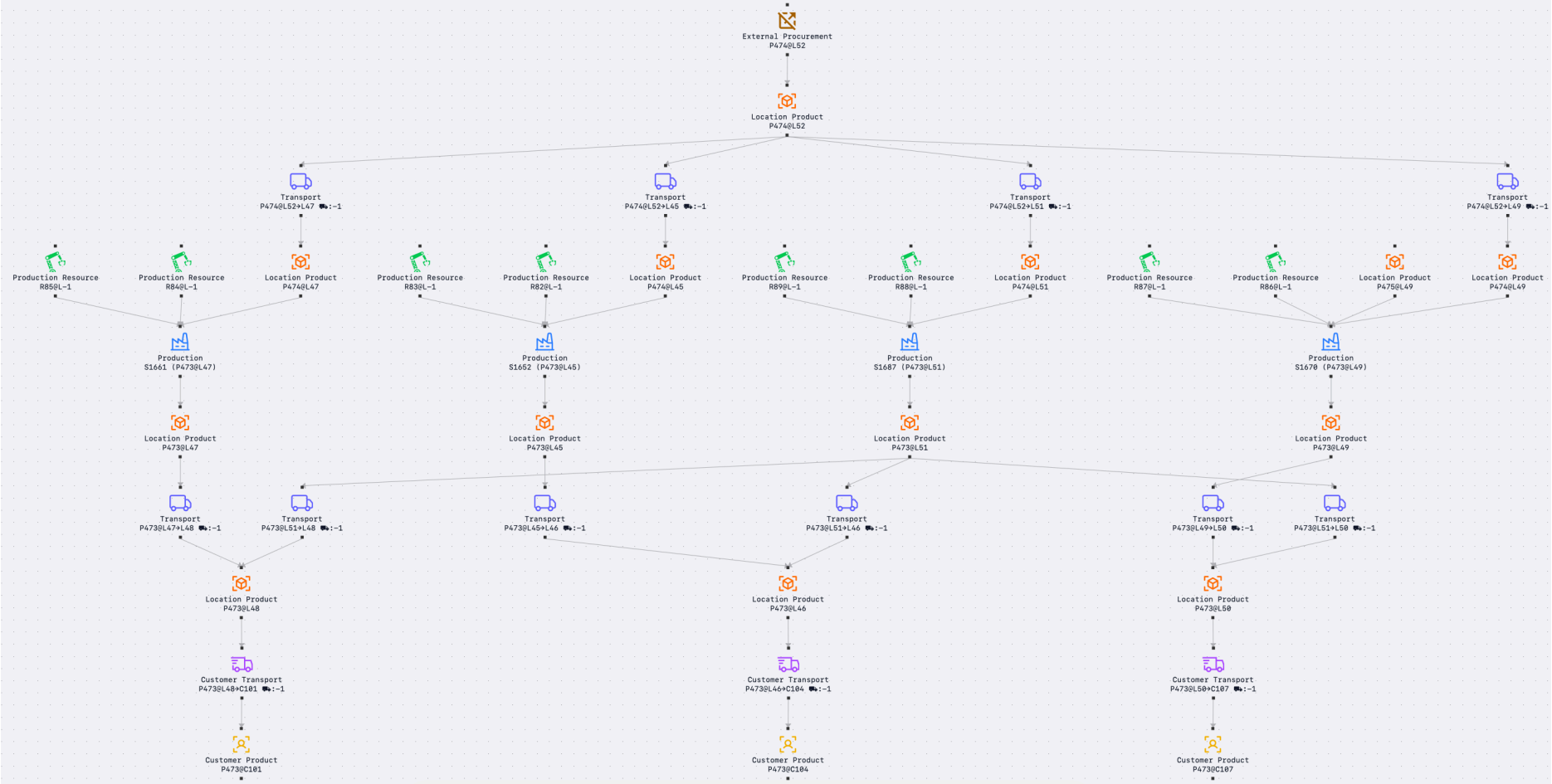


Figure 1: Example of a supply chain graph with different node types. Source: Internal SupplyChainViewer.

For the purposes of this thesis, the standard optimization pipeline in SAP *IBP* is modeled as:

$$\text{Input Tables} \rightarrow \text{Object-oriented Model} \rightarrow \text{MILP Model} \rightarrow \text{Solver} \rightarrow \text{Solution} \quad (4.)$$

The relational input tables are used to build the object-oriented model which is then transformed into a *MILP* formulation. The *MILP* is then submitted to an external solver such as Gurobi or SCIP/SoPlex [12], [13]. Finally, the solver's solution is interpreted back into operational decisions, such as how much to produce, what to transport, and when to procure materials.

Separately, the supply chain graph $G = (V, E)$ serves as an auxiliary abstraction built from the same input tables. This graph representation provides structural insight useful for analysis and, as explored in this thesis, for decomposition strategies. When any form of structural decomposition in this thesis is applied, the graph is constructed from the input tables, partitioned into subgraphs and these subgraphs guide how to filter or modify the original input data to create subproblems (see Section 6.2). Each subproblem's modified input data is then transformed into a *MILP* and solved.

This pipeline architecture enables SAP *IBP* to leverage external *MILP* solvers while maintaining a domain-specific interface for supply chain planners [4].

2.2.2 Mathematical Model Formulation

SAP *IBP* formulates the supply chain planning problem as a *MILP* containing variables, constraints and costs [14], [15].

The *MILP* formulation introduces decision variables that correspond to operational quantities across all time periods in the planning horizon [14]. Let $T = \{1, 2, ..., n\}$ be the discrete time periods over which decisions are made, called planning periods. Examples of the primary decision variables are:

- $\text{VO}_{o,s}(t)$ – the number of production lots of model o started at location s in period t
- $\text{VT}_{e,p}(t)$ – the quantity of material p transported on edge e in period t
- $\text{VS}_{s,p}(t)$ – the inventory level of material p at location s at the end of period t
- $\text{VL}_{d}(t)$ – the quantity delivered for demand d in period t

Various business requirements can introduce integer or binary variables into the formulation. For example, when lot sizes are active, production can only occur in discrete multiples of the lot size L_o, requiring $\text{VO}_{o,s}(t) \in \mathbb{Z}$. The output of the operation is then

given by $L_o \cdot \mathrm{VO}_{o,s}(t)$. Such integrality requirements make the problem a *MILP* rather than a pure *LP*.

The objective function minimizes total supply chain costs, balancing operational costs against delivery penalties [2]:

$$\begin{aligned}\min \sum_{t \in T} \Bigg[& \sum_{s \in S} \sum_{o \in O_s} \mathrm{CO}_{o,s} \cdot L_o \cdot \mathrm{VO}_{o,s}(t) + \sum_{s \in S} \sum_{p \in \mathrm{PS}_s} \mathrm{CS}_{s,p} \cdot \mathrm{VS}_{s,p}(t) \\ & + \sum_{e \in E} \sum_{p \in \mathrm{PE}_e} \mathrm{CT}_{e,p} \cdot \mathrm{VT}_{e,p}(t) \\ & + \sum_{d \in D} \mathrm{CD}_d \cdot (\mathrm{DQ}_d(t) - \mathrm{VL}_d(t)) \Bigg]\end{aligned} \tag{5.}$$

where $\mathrm{CO}_{o,s}$ denotes production costs per unit of production model o at location s; $\mathrm{CS}_{s,p}$ represents inventory holding costs; $\mathrm{CT}_{e,p}$ captures transportation costs per unit on edge e and CD_d is the penalty for each unit of unfulfilled demand d.

The non-delivery penalty term ensures that the model prefers fulfilling demand when feasible, with the penalty magnitude reflecting the business cost of missed deliveries.

The *MILP* formulation enforces physical and operational constraints that link the decision variables. The following presents the constraints most relevant for understanding the decomposition approach developed in this thesis:

Location-Product Constraints. For each location-product node, the inventory balance ensures that stock changes reflect all inflows and outflows:

$$\begin{aligned}\mathrm{VS}_{s,p}(t) = \mathrm{VS}_{s,p}(t-1) & + \sum_{o \in O_s} \mathrm{PR}_{o,s,p} \cdot \mathrm{VO}_{o,s}(t) \\ & + \sum_{e \in E_s} \mathrm{VT}_{e,p}(t) - \sum_{e \in E^s} \mathrm{VT}_{e,p}(t) - \mathrm{VL}_d(t)\end{aligned} \tag{6.}$$

where $\mathrm{PR}_{o,s,p}$ is the *BOM* coefficient representing the quantity of material p produced (positive) or consumed (negative) by production model o at location s. Location-product nodes contribute inventory holding costs $\mathrm{CS}_{s,p} \cdot \mathrm{VS}_{s,p}(t)$ to the objective.

Resource Constraints. Resource nodes impose shared capacity limits across multiple operations. Three types of resources exist: production resources constrain production operations, transport resources constrain transport operations and storage resources constrain inventory levels at locations. All three follow the same constraint structure. The resource consumption is computed directly in the capacity constraint by sum-

ming used quantities multiplied by resource requirement coefficients. For production resources:

$$\sum_{o \in O_{s_r}} \mathrm{RR}_{o,r} \cdot \mathrm{VO}_{o,s}(t) \leq \mathrm{RC}_r(t) \tag{7.}$$

where $\mathrm{RR}_{o,r}$ is the resource requirement for production model o on resource r and $\mathrm{RC}_r(t)$ is the capacity of resource r. Transport and storage resources follow analogously, with their respective variables VT and VS replacing VO. This constraint couples multiple operational decisions through shared capacity. Resource nodes do not directly contribute to the objective but constrain feasible solutions.

Non-Negativity. All quantity variables must be non-negative:

$$\mathrm{VO}_{o,s}(t), \mathrm{VT}_{e,p}(t), \mathrm{VS}_{s,p}(t), \mathrm{VL}_d(t) \geq 0 \tag{8.}$$

The complete mathematical formulation is not presented here, as it is beyond the scope of this thesis and would not contribute to the discussion of the proposed decomposition approach anyway.

3 Related Work

Mathematical decomposition techniques for mixed-integer linear programming are well established within the optimization community. Methods such as Benders decomposition [16], Lagrangian relaxation [17], [18], [19], [20] and Dantzig-Wolfe decomposition [5] operate at the solver level by exploiting algebraic structure in the constraint matrix [21], [22]. These techniques are domain-agnostic, applying generically to any *MILP* formulation without leveraging knowledge about what the variables and constraints represent. A related research question for supply chain optimization is how to decompose problems at the domain level, exploiting the network structure that defines how materials flow from suppliers through production stages to customers.

SAP Integrated Business Planning incorporates heuristic decomposition approaches to manage the computational complexity of supply network planning [23], [24]. Sandner [14] provides the foundational reference for SAP's decomposition strategies. These methods build upon classical decomposition theory [25] but adapt it for the practical constraints of mixed-integer formulations where exact methods cannot be directly applied due to integrality requirements [26], [27]. Four primary decomposition strategies are implemented: Priority Decomposition, Time Decomposition, Resource Decomposition and Product Decomposition. Priority Decomposition partitions demands by priority class, solving subproblems in descending order to ensure high-priority demands claim capacities first. Time Decomposition partitions along the temporal dimension, taking subsets of time buckets successively into the planning window while aggregating future periods into coarser buckets [28]. This rolling horizon approach relates conceptually to the multi-period planning structures studied in operations research generally [29]. Resource Decomposition partitions the problem based on subsets of resources, grouping production models and materials that depend on the selected resources [14]. Product Decomposition partitions along the material dimension, drawing on concepts from decentralized planning [30]. A preprocessing phase analyzes the material dependency graph to find connected components and resources are allocated across subproblems using a first-come, first-served strategy. The key structural characteristic is that Product Decomposition cuts horizontally through the supply network, following the direction of material flow [14].

The approach most similar to this thesis involves cutting through the material flow and solving different stages independently. Stage-wise decomposition for supply chains was pioneered by Clark and Scarf [31], who introduced echelon-based decomposition for

stochastic inventory systems, solving each stage separately through a single backward pass from downstream to upstream. Coupling between stages is handled through penalty terms in the cost function, achieving exact optimality for serial structures. This foundational work was extended in various directions [32], [33], [34], [35], but these results apply to stochastic dynamic programming for pure inventory problems rather than *MILP*.

Multi-echelon decomposition approaches that partition networks along functional or organizational boundaries face the challenge of coordinating the resulting subproblems [36]. Barbarosoglu and Ozgur [7] decomposed production-distribution planning into a two-level hierarchy, with coupling handled through Lagrangian relaxation where multipliers serve as price signals updated via subgradient optimization. This approach requires multiple iterations and provides no direct feasibility guarantee. The work of Lidestam and Ronnqvist [8] provides a particularly instructive case study. They decompose a supply chain into upstream procurement-production and downstream distribution subproblems, with flow variables duplicated at the boundary and Lagrangian multipliers penalizing discrepancies. Their computational experiments reveal the limitation that even after ten iterations, solutions frequently remain infeasible or exhibit gaps ranging from approximately one percent to over seventy percent, with several instances producing no feasible solution at all.

The dominant coordination mechanism in the literature is Lagrangian-based price coordination [7], [8], [19], [20], [28]. Lagrange multipliers are iteratively updated through subgradient optimization or bundle methods. These methods share common limitations as subproblems must be solved multiple times as coordination signals converge and solutions to relaxed subproblems may violate coupling constraints that were relaxed, requiring heuristic repair procedures whose success cannot be assured [22].

Existing domain-level decomposition approaches thus share the limitations that they require iterative coordination between subproblems and they provide no inherent feasibility guarantee. This thesis addresses this research gap with a graph-structural decomposition approach that exploits supply chain topology to coordinate subproblems through quantity-based coupling in a single up-and-down pass. Instead of price signals requiring iterative adjustment, the approach enforces hard flow constraints to maintain feasibility and uses objective function penalties to drive towards better solutions. This eliminates the iteration requirement while guaranteeing that the composed solution satisfies all constraints across subnetwork boundaries.

4 Methodology

This chapter describes the research methodology employed in this thesis and the evaluation design used to assess the proposed decomposition approach. The methodology follows the *Design Science Research (DSR)* paradigm, which is well-suited for developing and evaluating artifacts that address practical problems in information systems.

4.1 Research Approach

This thesis follows the *DSR* methodology, a research paradigm focused on creating and evaluating artifacts to solve identified organizational problems [37]. DSR is particularly appropriate for this work because the primary goal is to develop a practical solution that addresses the computational challenges of large-scale supply chain optimization.

The artifact developed in this thesis is a decomposition framework for supply chain networks modeled as *MILP* problems. This framework partitions complex supply networks into smaller subnetworks that can be solved independently, with the goal of reducing computational complexity while maintaining solution feasibility.

The DSR process in this thesis follows four main phases:

1. **Problem Identification and Motivation:** The limitations of monolithic *MILP* solving for enterprise-scale supply chains are identified and analyzed. This reasoning shows the practical relevance and necessity of a decomposition approach.
2. **Objectives Definition:** Clear objectives are defined based on the research questions: preserving solution feasibility (RQ1), comparing solution quality and computational performance against monolithic solving (RQ2) and enabling previously unsolvable problems to be solved (RQ3).
3. **Design and Development:** The decomposition framework is designed and implemented, including graph-based modeling, the decomposition strategy and the two-phase solving approach.
4. **Evaluation:** The artifact is evaluated against defined metrics using real-world benchmark scenarios, comparing performance against a monolithic baseline.

4.2 Evaluation Design

The evaluation compares the proposed decomposition approach against the monolithic baseline used in SAP *IBP*. In the baseline, the entire supply network is formulated and solved as a single *MILP*. For each scenario, both approaches solve the same optimization problem under identical conditions, with the same time limit, the same solver and the same parameter settings. This controlled setup isolates the effect of decomposition, allowing direct attribution of performance differences to the approach itself.

4.2.1 Metrics

The evaluation uses two primary metrics that represent the fundamental trade-off in optimization [2].

Runtime measures the solver time required to solve the optimization problem, excluding preprocessing and decomposition overhead. This focus on pure solver time is deliberate, since the primary research interest lies in understanding how decomposition affects the complexity of the underlying optimization problems themselves. By measuring only the time spent in the *MILP* solver, the evaluation captures the impact of partitioning large problems into smaller subproblems. Additionally, the current implementation introduces overhead due to architectural design decisions that would not be present in a production integration within SAP *IBP* (see Section 6), where decomposition logic could be tightly integrated with the existing optimization pipeline. Comparing total runtimes would confuse this implementation-specific overhead with the actual optimization benefits, obscuring the true potential of the decomposition approach. Moreover, the decomposition overhead is expected to be negligible for real customer scenarios with substantial solve times, since all employed algorithms are graph algorithms that scale as $O(V + E)$ and can be implemented very efficiently.

Solution Quality is measured by the objective function value of the resulting solution. In supply chain optimization, the objective function minimizes the total costs. Lower objective values indicate better solutions.

To ensure fair comparison between the decomposition approach and the monolithic baseline, both runs receive the same runtime budget, which corresponds to the operational time limit configured by the customer company for that scenario. The evaluation then compares the solution quality achieved within this budget. If one approach solves the problem to proven optimality before the time limit, the runtime budget for the

other approach is reduced to match this shorter time, ensuring a fair comparison at the point of optimality. If a run fails to find any feasible solution within the given time, its solution quality is reported as infeasible. In such cases, any feasible solution found by the other approach represents a clear improvement, regardless of its objective value.

4.2.2 Benchmark Scenarios

The evaluation uses 13 real customer scenarios from SAP *IBP*, deliberately selected to represent diverse network structures and complexity characteristics. These scenarios are anonymized production data that show the variety of supply chain configurations in practice. The selection aims to identify which network characteristics make decomposition beneficial and which scenarios may not benefit from the approach. The scenario statistics are shown in Table 1. Table 2 assigns each scenario a characteristic label describing its primary structural feature. While not comprehensive, these labels highlight distinguishing properties that allow pattern recognition and enable systematic analysis of which network characteristics benefit from decomposition.

Attribute	**S1**	**S2**	**S3**	**S4**	**S5**	**S6**
Graph Attributes:						
Total nodes	38,536	976	21,097	1,047,357	1,042,863	234,061
Loc-Prods	18,145	203	7,609	185,608	275,932	26,862
Productions	10,088	119	1,433	55,004	21,472	1,666
Transports	6,701	262	7,879	796,557	525,009	132,360
Customer-Prods	2,717	149	4,112	9,621	219,890	72,656
Procurements	402	236	4	567	165	474
Resources	483	7	60	0	395	43
Monolithic Model:						
Variables	5,276,857	66,361	146,810	212,315,069	9,580,634	2,046,469
Discrete Vars	102,480	6,644	43,349	4,299,160	1,058,220	13,027
Constraints	2,694,876	33,117	60,429	54,179,932	3,299,673	654,314

Attribute	**S7**	**S8**	**S9**	**S10**	**S11**	**S12**	**S13**
Graph Attributes:							
Total nodes	481,061	234,039	365	365	116,432	233,169	2,215
Loc-Prods	70,212	26,862	140	140	54,981	26,861	245
Productions	40,667	1,666	39	39	13,868	1,666	89
Transports	199,813	132,357	121	121	36,606	131,489	237
Customer-Prods	166,006	72,656	28	28	5,797	72,656	1,626
Procurements	2,327	455	20	20	5,051	454	6
Resources	2,036	43	17	17	129	43	12
Monolithic Model:							
Variables	6,109,827	6,597,019	38,396	47,859	1,276,677	7,016,374	100,972
Discrete Vars	181,711	59,224	6,372	25,268	578,864	142,924	5,297
Constraints	3,057,346	2,107,145	17,450	17,450	438,907	2,476,997	36,492

Table 1: Benchmark scenario characteristics

Scenario	**Characteristic**
S1	Medium-sized, Production-dominated
S2	Medium-sized, balanced structure, few resources
S3	Few procurements, production scenario
S4	No resources, very large
S5	High resource sharing, low procurement to consumer ratio
S6	Large production compared to procurement and distribution
S7	Large-scale, high resource count
S8	Few productions compared to transports
S9	Small, stage-aligned, balanced complexity
S10	Small, stage-aligned, same as S9 with extended horizon
S11	Very large, procurement to consumer ratio of 1
S12	Very large
S13	Production scenario, minimal procurement

Table 2: Scenario structural characteristics

5 Decomposition Framework Design

This chapter presents the design of a decomposition framework that partitions supply chain networks into smaller subnetworks, solves them sequentially with coordinated information exchange and combines their solutions into a feasible global solution. To achieve the approach of partitioning and solving subnetworks, the framework consists of several key components:

- A **graph-based representation** of supply chain networks that captures material flow and enables structural analysis
- A **decomposition strategy** for partitioning the network along the natural supply chain stages
- **Link nodes and virtual constraints** for connecting subnetworks at their boundaries
- **Constraint handling mechanisms** for shared resources that span multiple subnetworks
- A **two-phase solving approach** consisting of a bottom-up demand propagation phase and a top-down supply allocation phase
- **Aggregation heuristics** for propagating costs, lead time and capacity information across subnetwork boundaries
- A **solution combination process** that merges partial solutions into a feasible global solution

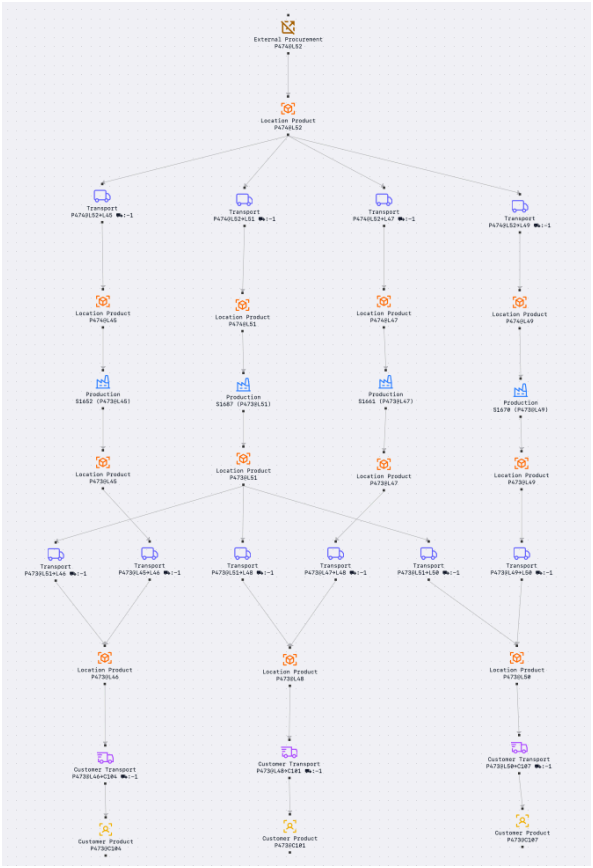
5.1 Graph-based Problem Modeling

The starting point of the framework is a supply chain defined by master data tables, which can be directly transformed into a *MILP* formulation and solved monolithically, i.e., as a single unified problem, using the standard optimization service. To enable decomposition, a directed graph $G = (V, E)$ is constructed as a structural abstraction from these tables, as described in Section 2.2.1. This graph-based representation captures the flow of materials through the network, defined by the master data tables described in Section 2.2.1. Unlike simple tree structures, supply chain graphs are multi-source, multi-sink and can contain cycles. Cycles in practical scenarios arise through closed-loop supply chains involving recycling, rework processes or side-outputs [38]. The presence of cycles has important implications for the decomposition approach discussed in subsequent sections.

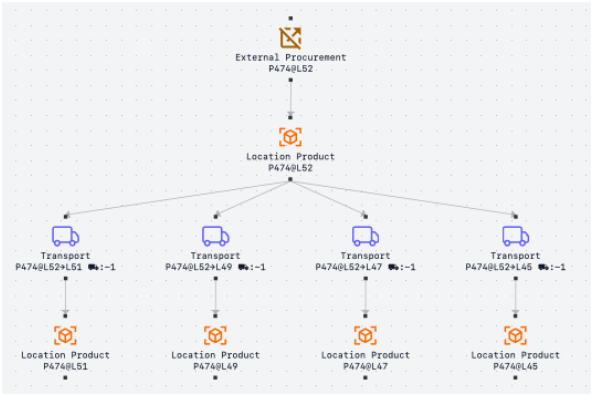
Each node $v \in V$ represents an operational entity in the supply chain and carries a type attribute corresponding to the node categories introduced in Section 2.2.1. Edges $e \in E$ represent material flow relationships and resource dependencies.

5.2 Decomposition

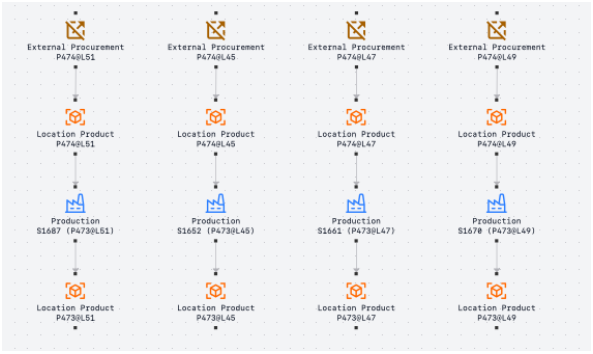
The decomposition partitions the supply chain graph $G = (V, E)$ into k disjoint subgraphs $\{G_0, G_1, ..., G_{k-1}\}$ based on the position of nodes within the supply chain. The goal is to create subproblems that can be solved sequentially, while each subnetwork represents an actual stage in a supply chain, e.g. Procurement, Production or Distribution. Figure 2 shows the target state of this decomposition process. The figure shows the several concepts that are now formally defined throughout this section.



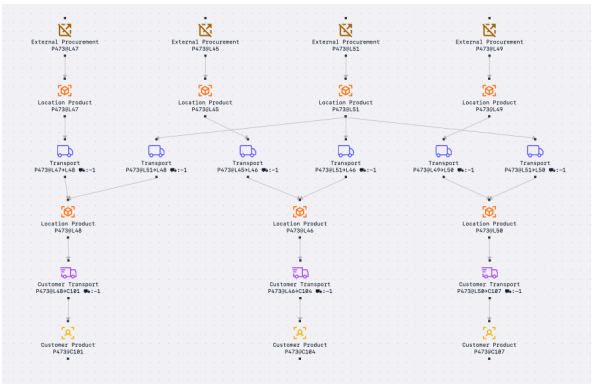
(a) Original supply chain graph



(b) Subnetwork 0: Procurement



(c) Subnetwork 1: Production



(d) Subnetwork 2: Distribution

Figure 2: Decomposition of a supply chain graph into three subnetworks with link nodes and virtual procurements. Source: Internal SupplyChainViewer.

5.2.1 Cycles

The stage assignment and partitioning relies on the relative position of nodes and well-defined up- and downstream relationships. Cyclic dependencies invalidate this assumption, making the assignment ambiguous. Furthermore, partitioning a cycle across multiple subnetworks introduces circular dependencies between them, preventing the

subnetworks from being ordered for sequential optimization. Consequently, cycles must remain entirely within a single subnetwork and may not be split during partitioning.

To achieve this, the decomposition operates on the condensation $G_C = (V_C, E_C)$ of the network rather than on G directly. The condensation is constructed by first identifying all *Strongly Connected Components (SCCs)*, then contracting each *SCC* into a single meta-node. An edge exists between two meta-nodes in G_C if and only if an edge exists between their corresponding components in G. Since all cycles are absorbed into the contracted nodes, the condensation is always a *Directed Acyclic Graph (DAG)*, enabling topological traversal. The stage classification therefore assigns stages to meta-nodes (SCCs) rather than individual nodes and all nodes within an *SCC* inherit the same stage from their containing meta-node.

5.2.2 Classification

Each node $v \in V_C$ is assigned a supply chain stage $s(v) \in \{1, 2, 3, 4, 5\}$ based on its relative position inside the network. The five stages reflect the typical phases of material flow in supply chains: procurement from suppliers, pre-processing, production/ transformation, post-processing of finished goods and distribution to customers. This classification provides the finest meaningful granularity and coarser decompositions can be achieved by grouping stages together, as described in the following section. Table 3 summarizes the five stages and their criteria and Figure 3 shows an example classification of nodes.

Stage	**Name**	**Criterion**
1	Sourced	Is procurement or input from procurement
2	Pre-Production	Upstream of all production, no direct procurement
3	Intermediate	Contains production or lies between production stages
4	Finished	Downstream of all production, no customer connection
5	Distribution	Is customer node or output to customer node

Table 3: Supply chain stage classification

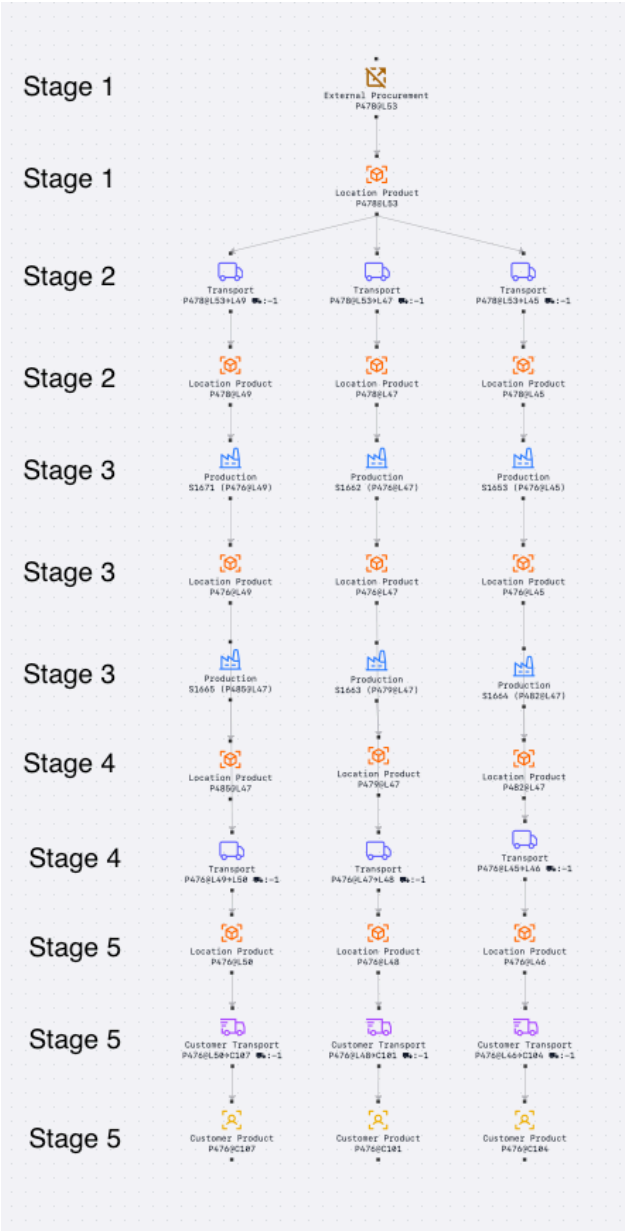


Figure 3: Example stage classification of nodes. Source: Internal SupplyChainViewer.

The formal definition of the stage sets uses the position of nodes relative to production. Let $\tau : V_C \to \{\text{Proc}, \text{Prod}, \text{Cust}\}$ denote the node type function. For singleton meta-nodes, i.e., SCCs containing a single original node, τ maps to the type of that node. For multi-node SCCs, an SCC may contain nodes of different types. In this case, τ maps to the dominant type according to the priority order Proc $>$ Prod $>$ Cust. Let $P = \{v \in V_C : \tau(v) = \text{Prod}\}$ be the set of production meta-nodes. Using reachability analysis on G_C, where $v \xrightarrow{*} p$ denotes that v can reach p via a directed path, define:

- $U_P = \left\{v \in V_C : \exists p \in P, v \xrightarrow{*} p\right\}$: nodes that are upstream of production (i.e., nodes from which at least one production node is reachable along the directed path)

- $D_P = \left\{v \in V_C : \exists p \in P, p \xrightarrow{*} v\right\}$: nodes that are downstream of production (i.e., nodes that are reachable from at least one production node along the directed path)

Note that every node can reach itself, meaning $p \xrightarrow{*} p$ holds for any node p. Consequently, all production nodes belong to both U_P and D_P.

Nodes in $U_P \setminus D_P$ are upstream of at least one production node but downstream of none (stages 1-2), nodes in $D_P \setminus U_P$ are downstream of at least one production node but upstream of none (stages 4-5) and nodes in $U_P \cap D_P$ lie between production stages (stage 3). Within these groups, the presence of direct procurement or customer edges determines the final stage assignment:

$$\begin{aligned} S_1 &= \{v \in U_P \setminus D_P : \tau(v) = \text{Proc} \vee \exists (u,v) \in E_C, \tau(u) = \text{Proc}\} \\ S_2 &= \{v \in U_P \setminus D_P : \tau(v) \neq \text{Proc} \wedge \neg\exists (u,v) \in E_C, \tau(u) = \text{Proc}\} \\ S_3 &= U_P \cap D_P \\ S_4 &= \{v \in D_P \setminus U_P : \tau(v) \neq \text{Cust} \wedge \neg\exists (v,w) \in E_C, \tau(w) = \text{Cust}\} \\ S_5 &= \{v \in D_P \setminus U_P : \tau(v) = \text{Cust} \vee \exists (v,w) \in E_C, \tau(w) = \text{Cust}\} \end{aligned} \tag{9.}$$

Let $R = V_C \setminus (U_P \cup D_P)$ be the set of nodes without any production in their connected component. Let $R_1 = \{v \in R : \tau(v) = \text{Proc} \vee \exists (u,v) \in E_C, \tau(u) = \text{Proc}\}$ be the subset that are procurement nodes or have direct procurement input and $R_5 = \{v \in R : \tau(v) = \text{Cust} \vee \exists (v,w) \in E_C, \tau(w) = \text{Cust}\}$ the subset that are customer nodes or have direct customer output. Then:

$$\begin{aligned} S_1 &:= S_1 \cup R_1 \\ S_5 &:= S_5 \cup (R_5 \setminus R_1) \\ S_4 &:= S_4 \cup (R \setminus R_1 \setminus R_5) \end{aligned} \tag{10.}$$

Following these definitions every node is assigned correctly to its stage in the supply chain as described in Table 3. These sets S_1 to S_5 are pairwise disjoint and partition V_C, which is proven in Appendix 1. This property is essential for correctness, since it guarantees that each node is assigned to exactly one stage, preventing variables and constraints from being duplicated or lost during creation of the subgraphs and then subproblems from the stages. Resource nodes are excluded from the stage assignment and partitioning process beforehand, as their capacities may be shared across multiple subnetworks and require coordinated handling across partition boundaries. Their treatment is detailed in Section 5.5.

5.2.3 Subgraph Construction

A stage grouping $\mathcal{S} = \{S_1, S_2, ..., S_k\}$ partitions the five stages into k groups, where each group becomes one subnetwork. To partition the supply chain into the natural stages of procurement, production and distribution, the grouping $\mathcal{S} = \{\{1, 2\}, \{3\}, \{4, 5\}\}$ is used in this thesis, which creates three subnetworks corresponding to these stages.

Given the stage function $s : V \to \{1, ..., 5\}$, mapping a node to its stage and the grouping $\mathcal{S}$, the node assignment function $\sigma : V \to \{1, ..., k\}$ maps each node to its subnetwork:

$$\sigma(v) = i \quad \text{where} \quad s(v) \in S_i \tag{11.}$$

The subgraph of the subnetwork i, $G_i = (V_i, E_i)$ contains:

$$V_i = \{v \in V : \sigma(v) = i\} \tag{12.}$$

$$E_i = \{(u, v) \in E : u \in V_i \wedge v \in V_i\} \tag{13.}$$

Every edge $(u, v) \in E$ falls into exactly one of two categories: either both endpoints belong to the same subnetwork ($\sigma(u) = \sigma(v)$), in which case the edge is included in E_i, or the endpoints belong to different subnetworks ($\sigma(u) \neq \sigma(v)$), making it a cut edge. Cut edges require special handling through cut harmonization, but no edge is lost in the decomposition.

5.2.4 Cut Harmonization

Cut edges $(u, v) \in E$ where $\sigma(u) \neq \sigma(v)$ represent material flow between subnetworks. These edges cannot simply be removed, because they are the coupling between the subnetworks. To enable the exchange of materials across partition boundaries, the framework introduces “link nodes”. Link nodes are designated location-product nodes that serve as the interface between adjacent subnetworks.

By the structure of supply chain graphs (see Section 2.2.1), every edge in the material flow connects a source-of-supply node (Production, Transport, Procurement) to a location-product node, or vice versa. Customer nodes form a special case, but a customer transport node and a customer product node are always assigned to the same stage, so they cannot appear on cut edges. Consequently, every cut edge has at least one location-product node as an endpoint. These location-product nodes on cut edges form the set of link nodes $\mathcal{L}$:

$$\mathcal{L} = \{\ell \in V : \ell \text{ is a location-product node} \wedge \exists (u, v) \in E, \sigma(u) \neq \sigma(v), \ell \in \{u, v\}\} \tag{14.}$$

Each link node $\ell \in \mathcal{L}$ is replicated in both adjacent subnetworks. Consider an edge from production node $p \in G_i$ to link node $\ell \in G_j$. The node ℓ is added to G_i and an edge from p to ℓ is added in G_i. The link node ℓ now serves as the interface coupling between the subnetworks.

For each link node $\ell \in \mathcal{L}$ in the downstream subnetwork G_j, a virtual external procurement node v_ℓ^{virt} is created and connected to ℓ. This virtual node represents the supply that will flow from the upstream subnetwork to the downstream subnetwork. An example of this setup can be seen in Figure 2. Therefore, when solving the downstream subnetwork individually this virtual procurement node v_ℓ^{virt} gets treated as a supply point and the amount of the requested supply of the downstream subnetwork can be propagated to the upstream subnetwork (see Section 5.3).

5.2.5 Constraint Deduplication and Feasibility

The replication of link nodes introduces the challenge that the same location-product now appears in two subnetworks, which would lead to duplicated constraints and variables and therefore capacities if not handled correctly. In the original monolithic formulation, each location-product has exactly one inventory balance constraint and one inventory variable. After replication, just generating constraints for both copies would double-count inventory capacity, initial stock and storage costs.

To preserve feasibility, the framework must assign all inventory-related constraints exclusively to one copy of each link node $\ell \in \mathcal{L}$. While either copy could serve this role, the framework assigns them to the upstream copy in subnetwork G_i by convention. The downstream copy (in subnetwork G_j) is marked as "non-storable", eliminating its inventory constraints and storage variables. The downstream copy thus functions as a pure flow-through point. This design ensures that the combined solution across subnetworks will maintain feasibility with respect to the original formulation.

5.3 Two-Phase Solve Approach

The coordination of subproblems across the decomposed network requires a systematic approach that respects the dependencies between subnetworks while ensuring solution feasibility. In contrast to iterative decomposition methods that require multiple passes to achieve convergence, this newly proposed approach consists of exactly two traversals: a bottom-up phase that propagates demand information from distribution toward pro-

curement, followed by a top-down phase that allocates actual supplies from procurement toward distribution.

The bottom-up phase solves each subnetwork assuming unlimited supply at its virtual procurement cut points, to determine the demand requirements that should be satisfied by the upstream subnetwork. Subsequently, the top-down phase resolves each subnetwork with the actual supply capacities determined by its upstream subnetworks, yielding a feasible allocation solution throughout the network. This two-pass approach eliminates the convergence uncertainty of iterative approaches while ensuring that, provided all subproblems obtain feasible solutions, the combined solution satisfies the original constraints.

5.3.1 Meta Graph Construction

While handling cut-edges, a meta graph $M = (\mathcal{V}_M, \mathcal{E}_M)$ is constructed, that captures the dependencies between the k subnetworks:

$$\mathcal{V}_M = \{0, 1, ..., k-1\} \tag{15.}$$

$$\mathcal{E}_M = \{(i, j, \ell) : \exists (u, v) \in E \text{ with } \sigma(u) = i, \sigma(v) = j, \ell \in \mathcal{L} \cap \{u, v\}\} \tag{16.}$$

The meta graph is a multi-graph allowing multiple edges between the same pair of subnetworks, one per distinct link node. The meta graph defines the solving order for both phases. The reverse of a topological ordering of M yields a sequence in which each subnetwork appears only after all of its successors, establishing the correct execution order for the solve of the corresponding subproblems in the bottom-up phase. Conversely, the top-down phase requires all upstream supplies to be determined before a subnetwork can be solved. The topological order therefore provides the correct execution sequence for this case. Figure 4 shows an example meta graph which would result in $(2, 1, 0)$ for the bottom-up solve and $(0, 1, 2)$ for the top-down solve.

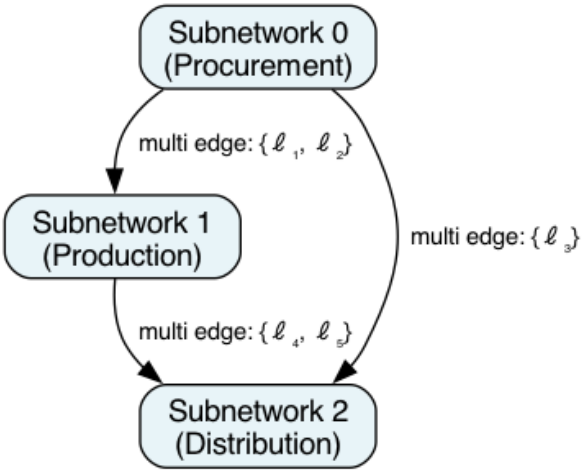


Figure 4: Example meta graph M for three subnetworks

5.3.2 Demand Propagation (Bottom-Up Phase)

The bottom-up phase determines how much material each subnetwork requires from its upstream neighbors. Subnetworks are solved in reverse topological order, starting from distribution (closest to customers) and proceeding toward procurement. Each subnetwork G_j is solved as an independent scenario with virtual procurement nodes representing the possible, currently infinite supply from upstream subnetworks. This means the modified constraint for link node ℓ in the subnetwork G_j becomes:

$$\mathrm{VS}_\ell(t) = \mathrm{VS}_\ell(t-1) + \mathrm{VP}_\ell(t) + ..., \tag{17.}$$

with decision variable $\mathrm{VP}_\ell(t)$ that represents the procurement from the upper subnetwork, allowing the solver to acquire material at this location, that is propagated to the supplying subnetwork in the following way. Let $\mathrm{VP}_v^{\mathrm{virt}}(t)$ denote the planned procurement quantity at virtual node v by subnetwork G_j. This quantity represents the demand that subnetwork G_j requests from its upstream neighbor. After solving G_j, the demand is propagated to the upstream subnetwork G_i by creating a location demand at the corresponding boundary location-product node ℓ:

$$\mathrm{DQ}_\ell(t) = \mathrm{VP}_{v_\ell^{\mathrm{virt}}}(t) \tag{18.}$$

where $\mathrm{DQ}_\ell(t)$ is the demand at location-product ℓ in period t in subnetwork G_i and $\mathrm{VP}_{v_\ell^{\mathrm{virt}}}(t)$ is the external procurement quantity from the virtual procurement node connected to ℓ in G_j. This demand is added to the upstream subnetwork as a demand target with constraint $\mathrm{VP}_\ell(t) \leq \mathrm{DQ}_\ell(t)$. Then, the solve of subproblem G_i determines the optimal delivered quantity $\mathrm{VL}_\ell(t)$, which appears both in the inventory balance as an outflow and in the objective function with a penalty term $\mathrm{CD}_\ell \cdot (\mathrm{DQ}_\ell(t) - \mathrm{VL}_\ell(t))$

for any unfulfilled portion. The non-delivery penalty CD_ℓ is derived from an aggregated non-delivery cost, described in Section 5.4.4.

Since the virtual procurement nodes have unlimited capacity during the bottom-up phase, the solver determines ideal demand requirements without supply constraints, propagating demands backward through the supply chain from customer demands to procurements. This may result in demands that exceed what upstream subnetworks can actually provide. When an upstream subnetwork cannot fully satisfy a downstream demand, the downstream subnetwork's solution becomes temporarily infeasible. The top-down phase corrects this by re-solving each subnetwork with the actual supplies delivered by its upstream neighbors, ensuring that, provided all subproblems obtain feasible solutions, the combined solution satisfies the original cross-subnetwork constraints.

5.3.3 Supply Propagation (Top-Down Phase)

The top-down phase then allocates actual supplies based on the supply decisions of upstream subnetworks. Subnetworks are solved in topological order, starting from procurement and proceeding toward distribution. The first subnetwork retains its bottom-up solution unchanged, as it has no upstream dependencies which could need correction.

For each subsequent subnetwork G_j, the virtual procurement nodes are replaced with fixed supply quantities derived from the upstream solutions. $\mathrm{VL}_\ell^{G_i}(t)$ denotes the quantity actually delivered to location-product ℓ in period t in the upstream subnetwork G_i. The top-down phase injects this quantity as a constant term into the inventory balance constraint (Equation 6). The modified constraint for link node ℓ in the downstream Subnetwork G_j becomes:

$$\mathrm{VS}_\ell(t) = \mathrm{VS}_\ell(t-1) + \mathrm{VL}_\ell^{G_i}(t) + \ldots \tag{19.}$$

where $\mathrm{VL}_\ell(t)$ is now a fixed constant rather than a decision variable. This fixed supply term represents the actual material availability from the upstream subnetwork. The virtual external procurement rule for the boundary node is removed. The top-down solve may produce a different, worse solution compared to the bottom-up phase, since the actual supply may be less than the requested demand.

5.4 Link Node Heuristics

The subnetworks have no information about cost-structure or non-delivery penalties of neighboring subnetworks. While the stated design already produces valid solutions for any problem, the solutions can be improved by providing more information to the different subproblems. Therefore, the virtual procurement nodes introduced at cut edges are parameterized without access to actual upstream subproblem solutions. Each virtual procurement gets assigned four parameters:

Parameter	**Purpose**
Virtual marginal cost	Approximate cheaper upstream procurements
Leadtime aggregation	Prevent requesting before supply is possible
Maximum flow capacity	Identify dead ends that will never deliver
Virtual non-delivery penalty	Prioritize fulfillment of higher-priority demands

Table 4: Virtual procurement parameters

Computing exact values would necessitate solving the upstream subproblems first, defeating the purpose of decoupled optimization. The framework therefore employs conservative heuristic estimates that keep the solution space of the combined subnetwork solution the same, but restrict the solution spaces of the subproblems in the bottom-up phase, to get better virtual demand estimates. This means that the proposed heuristics can only improve the overall decomposition solution. The parameters in Table 4 are assigned to the virtual procurement nodes created for each link node and will then be transformed into constraints during the *MILP* formulation. The following sections describe each heuristic and how the computed values enter the subproblem formulation.

The aggregation heuristics must respect the structural semantics of different node types. Let $\text{pred}(v) = \{u \in V : (u, v) \in E\}$ denote the set of immediate predecessors of node v and let $\text{succ}(v) = \{w \in V : (v, w) \in E\}$ denote the set of immediate successors. Two distinct behaviors are distinguished based on the logical relationship between a node and its predecessors:

- **OR-nodes**: Location-product and customer-product nodes represent points where material can arrive from multiple alternative sources. One source would be enough to fulfill demand.
- **AND-nodes**: Production nodes require inputs from all predecessors simultaneously, consuming components according to the *BOM* coefficients $\text{PR}_{o,s,p}$ defined by the

production model, where negative values indicate consumed materials and positive values indicate produced materials.

All other node types have exactly one predecessor by definition.

5.4.1 Production Cost Aggregation

The virtual procurement nodes require cost parameters representing the expense of obtaining materials from upstream subnetworks. Without these parameters, the downstream optimization would treat upstream supply as cost-free, leading to over-procurement. Production cost aggregation computes an estimated procurement cost for each boundary product based on the accumulated costs along the upstream supply paths.

Let c_v denote the marginal cost of node v, representing the direct operational cost per unit on the operation (e.g. transport cost, production cost, procurement cost). The aggregation proceeds in topological order from sources to sinks. The aggregated cost c_v^{agg} is computed as:

$$c_v^{\text{agg}} = \begin{cases} c_v & \text{if } \text{pred}(v) = \emptyset \\ c_v + \min_{u \in \text{ pred}(v)} c_u^{\text{agg}} & \text{if } v \text{ is an OR-node} \\ c_v + \sum_{u \in \text{ pred}(v)} |\text{PR}_{(u,v)}| \cdot c_u^{\text{agg}} & \text{if } v \text{ is an AND-node} \end{cases} \quad 20.$$

For OR-nodes, the minimum reflects the optimizer's ability to choose the cheapest supplier. For AND-nodes, the sum scaled by *BOM* coefficients captures the total input cost. The aggregated cost constitutes an underestimate of the true procurement cost, as the minimum operator assumes the cheapest path is always available. Capacity constraints may force later selection of more expensive alternatives in practice, but for the purpose of guiding the optimization toward cheaper decisions, this approximation is sufficient. The computed c_v^{agg} is assigned as the procurement cost parameter of the virtual procurement node, entering the objective function as a per-unit cost term.

5.4.2 Lead Time Aggregation

Lead time l_v represents the number of periods required from the initiation of an operation at node v until the resulting material becomes available. When a subnetwork boundary is introduced, the downstream subnetwork loses visibility into upstream timing needs. The virtual procurement node must therefore carry an aggregate lead time reflecting the minimum time required for material to traverse the upstream network up to the point of the link node.

The aggregation proceeds in topological order. The aggregated lead time l_v^{agg} is computed as:

$$l_v^{\text{agg}} = \begin{cases} l_v & \text{if } \text{pred}(v) = \emptyset \\ \min_{u \in \text{ pred}(v)} l_u^{\text{agg}} + l_v & \text{if } v \text{ is an OR-node} \\ \max_{u \in \text{ pred}(v)} l_u^{\text{agg}} + l_v & \text{if } v \text{ is an AND-node} \end{cases} \quad 21.$$

For OR-nodes, the minimum selects the fastest alternative procurement path. For AND-nodes, the maximum identifies the critical path, since all inputs must be present before production can commence.

The computed l_v^{agg} at boundary nodes is assigned to virtual procurement nodes as their validity period. This aggregation provides an optimistic underestimate, since capacity constraints may still limit the flow in valid periods because of missing input materials.

5.4.3 Max Flow Aggregation

The max flow aggregation identifies "dead end" link nodes where no material can feasibly flow across the subnetwork boundary. Such dead ends arise when upstream production paths are structurally infeasible, for example due to disabled productions or missing input materials. Without identifying these dead ends, the downstream optimization would create virtual procurement nodes for link nodes that can never receive supply from the upstream subnetwork, leading to unfulfillable demand requests during the bottom-up phase and therefore worse solutions.

The aggregation proceeds in topological order from source nodes toward boundary nodes. Let f_v^{agg} denote the aggregated max flow at node v and let f_v^{own} denote the node's own capacity constraint (e.g. Maximum Transport, Maximum Production, Resource Capacity). The aggregated max flow is computed as:

$$f_v^{\text{agg}} = \begin{cases} f_v^{\text{own}} & \text{if } \text{pred}(v) = \emptyset \\ \sum_{u \in \text{ pred}(v)} f_u^{\text{agg}} & \text{if } v \text{ is an OR-node} \\ \min_{u \in \text{ pred}(v)} \frac{f_u^{\text{agg}}}{|\text{PR}_{(u,v)}|} & \text{if } v \text{ is an AND-node} \end{cases} \quad 22.$$

For OR-nodes, the sum reflects that any combination of sources can contribute to total flow. For AND-nodes, the minimum identifies the bottleneck input, scaled by the *BOM* coefficient to convert input capacity to output capacity. The final aggregated value incorporates any local resource constraints: $f_v^{\text{agg}} = \min(f_v^{\text{agg}}, f_v^{\text{own}})$.

When a link node ℓ has $f_\ell^{\text{agg}} = 0$ across all planning periods, it represents a structural dead end where no feasible flow can occur. In this case, the corresponding virtual

procurement node is entirely removed from the downstream subnetwork. This creates better demand estimates to the upper subnetworks, because it stops the downstream subnetwork from attempting to request supply through paths that can never deliver.

5.4.4 Non-Delivery Cost Aggregation

Non-delivery cost represents the penalty incurred when customer demand cannot be fulfilled. Virtual procurement nodes in upstream subnetworks have no visibility into the downstream demand structure they serve. Without propagating penalty information upstream, virtual link nodes cannot distinguish between demands of varying importance. The non-delivery cost aggregation propagates penalty costs backward from demand nodes to their upstream suppliers.

Unlike the other heuristics, this aggregation operates in reverse topological order, from sink nodes toward source nodes. Let CD_v denote the non-delivery penalty at a demand node v. The aggregated non-delivery cost $\mathrm{CD}_v^{\mathrm{agg}}$ is computed as:

$$\mathrm{CD}_v^{\mathrm{agg}} = \begin{cases} \mathrm{CD}_v & \text{if } \mathrm{succ}(v) = \emptyset \\ \mathrm{CD}_v + \frac{1}{|\mathrm{succ}(v)|} \sum_{w \in \mathrm{succ}(v)} \frac{\mathrm{CD}_w^{\mathrm{agg}}}{|\mathrm{PR}_{v,w}|} & \text{otherwise} \end{cases} \tag{23.}$$

The inverse *BOM* scaling accounts for material conversion ratios, since, when multiple units of a component produce one unit of output, failing to deliver one component unit affects, on average, only a fraction of the final product. The average over successors distributes the downstream penalty proportionally when a node feeds multiple paths.

This heuristic is inherently imprecise since it remains unknown which specific customer demand will ultimately be fulfilled by delivering a particular intermediate product. Furthermore, failing to deliver one component of a production process prevents the entire output from being produced, not merely the fraction implied by the *BOM* coefficient. Nevertheless, the aggregated penalty provides the solver with directional guidance to prioritize fulfilling demands with higher downstream criticality. The computed $\mathrm{CD}_v^{\mathrm{agg}}$ is assigned as the non-delivery penalty CD_ℓ of the virtual demand created at the link node, entering the objective function as a penalty term for unfulfilled virtual demand.

5.5 Inter-Subnetwork Constraint Handling

The graph partitioning described in the previous sections creates subnetworks that can be solved independently, but certain constraints in the original formulation cannot

be assigned to a single subproblem. As introduced in Section 2.2.2, resource nodes impose shared capacity limits across multiple operations through constraints of the form $\sum_{o \in O_{s_r}} \mathrm{RR}_{o,r} \cdot \mathrm{VO}_{o,s}(t) \leq \mathrm{RC}_r(t)$, where O_{s_r} is the set of production models using the resource's capacity. When operations from different subnetworks share the same resource, this constraint spans multiple subproblems and cannot be enforced locally. Without coordination, each subnetwork would assume access to the full resource capacity, leading to collective over-allocation that violates the original constraint. This section presents an approach for allocating shared resource capacity across subnetworks.

The equal split strategy provides a straightforward mechanism for distributing shared resource capacity across subnetworks. The capacity is divided equally among all sharing subnetworks, regardless of their actual requirements. This approach requires only structural information about which subnetworks reference each resource, without knowledge of consumption patterns or optimization outcomes.

Let r denote a shared resource with capacity $\mathrm{RC}_r(t)$ in period t and let $N_r = \{i_1, i_2, ..., i_m\}$ be the set of subnetwork indices that contain operations consuming r. The capacity allocated to each subnetwork $i \in N_r$ is:

$$\mathrm{RC}_r^{(i)}(t) = \frac{\mathrm{RC}_r(t)}{|N_r|} \tag{24.}$$

The equal split approach offers simplicity and guaranteed feasibility since each subnetwork receives a fraction of the original capacity and these fractions sum to one, the combined usage across all subnetworks cannot exceed the original constraint. However, this strategy can result in inefficient utilization when subnetworks have heterogeneous demands. For the sake of simplicity this approach is used throughout the experiments. It is expected that the solution quality can be improved by using more sophisticated approaches, like *First-Come, First-Served (FCFS)* or Preallocation. Possible designs are discussed in Section 9, but the implementation and evaluation remain future work.

5.6 Solution Combination

After the top-down phase completes, each subnetwork has produced a local solution. These solutions must be combined into a single solution for the original monolithic problem. The combination process addresses two categories of variables:

- unique variables that exist in exactly one subnetwork,
- link node variables at cut edges.

5.6.1 Unique Variables

The majority of decision variables in the original formulation appear in exactly one subnetwork. Decision variables for nodes that are not link nodes are directly adopted from their respective subnetwork solution. These variables require no special handling since their constraints are entirely contained within a single subnetwork.

5.6.2 Link Node Variables

As described in Section 5.2, link nodes $\ell \in \mathcal{L}$ are replicated across adjacent subnetworks during cut harmonization. The upstream copy in subnetwork G_i retains all inventory-related constraints and variables, while the downstream copy in G_j functions as a flow-through point with no inventory constraints.

During the two-phase solve, virtual demands and virtual procurements are introduced at link nodes to coordinate the subnetworks. In the combined solution, these virtual elements are removed. The upstream copy's solution provides all inventory and constraint-related values for the link node. The delivered quantity $\mathrm{VL}_\ell(t)$ from the upstream subnetwork becomes the actual inventory outflow to the downstream subnetwork, satisfying the original material inflow constraint without requiring an explicit demand variable in the combined solution:

$$\mathrm{VS}_{s,p}^{\text{combined}}(t) = \mathrm{VS}_{s,p}^{(i)}(t) \quad \text{for } \ell \in \mathcal{L} \tag{25.}$$

This ensures that the combined solution satisfies the original inventory balance constraints, because the decomposition assigns all inventory constraints exclusively to the upstream copy. The downstream copy has no inventory variable and constraint that would conflict.

The resource capacity splitting strategies ensure that each subnetwork respects its allocated capacity share. Since the allocated shares sum to at maximum the original capacity, the combined resource consumption cannot exceed the original resource constraint $\mathrm{RC}_r(t)$.

The combination step is straightforward because the top-down phase has already resolved all cross-boundary dependencies since each subnetwork was solved with the actual supplies provided by its upstream neighbors, ensuring that the combined solution is feasible with respect to the original formulation.

5.7 Runtime Allocation

When solving large-scale supply network instances, a total runtime budget is specified to limit computational cost. In the monolithic case, this budget is allocated to a single *MILP* solver invocation. Under decomposition, the total budget must be distributed across multiple subproblem solves. When dividing into k subnetworks the bottom-up phase needs k *MILP* solve runs and the top-down phase needs $k-1$ *MILP* solve runs, because the uppermost subnetwork retains its solution unchanged. This leads to $2k-1$ solve operations in total. A time distribution strategy needs to address that problems with fewer variables will find better solutions faster and might be solved to optimality before the allocated time expires. Bigger problems on the other hand might need more time to find even one possible solution, which is crucial to continue the whole solve, since when one problem does not find a solution, the whole run cannot be continued.

A complexity-weighted allocation strategy is therefore introduced, that distributes runtime proportionally to estimated subproblem difficulty. Since the exact model statistics are only available once the mathematical model is built, estimates depending on the graph nodes and their master data are used. Let n_i^{cont} denote the estimated number of continuous variables and n_i^{disc} the estimated number of discrete variables in subnetwork i. From these estimates, a heuristic complexity score C_i is computed using an internal formula employed by existing SAP decomposition modules. The exact calculation of C_i is not specified in this thesis, as the underlying heuristic is an internally developed measure that has not been formally validated as a general measure of computational complexity. The formula weights discrete variables more heavily than continuous ones, reflecting the exponential impact of integrality constraints on branch-and-bound search complexity. The estimations for counting discrete and continuous variables per node type are detailed in Appendix 2.

The complexity scores are computed once before solving begins and normalized to obtain time shares $s_i = \frac{C_i}{\sum_{j=0}^{k-1} C_j}$ for each subnetwork. However, the actual runtime allocation is performed dynamically during the solving process based on remaining subproblems and previously consumed time.

Let T_{rem} denote the remaining runtime budget and S_{rem} the sum of time shares for all unsolved subnetworks. When subnetwork i is about to be solved, its allocated runtime is:

$$t_i = T_{\text{rem}} \cdot \frac{s_i}{S_{\text{rem}}} \tag{26.}$$

After solving subnetwork i, let t_i^{used} denote its actual solver runtime. The remaining budget is updated as $T_{\text{rem}} := T_{\text{rem}} - t_i^{\text{used}}$ and subnetwork i's time share is removed from the remaining share pool: $S_{\text{rem}} := S_{\text{rem}} - s_i$. This dynamic reallocation ensures that any time saved by a subnetwork that solves faster than expected is redistributed proportionally among the remaining unsolved subnetworks according to their complexity shares.

To prevent excessively small allocations to low-complexity subnetworks, a minimum share threshold $s_{\min} = 0.1$ (10%) is enforced, ensuring each subnetwork receives at least 10% of the total budget regardless of its complexity. If a subnetwork's complexity-based share s_i falls below $s_{\min}$, it is raised to $s_{\min}$. The required additional share is taken proportionally from subnetworks whose shares exceed $s_{\min}$. Specifically, let $D = \sum_{s_i < s_{\min}} (s_{\min} - s_i)$ be the total deficit and $S = \sum_{s_i \geq s_{\min}} (s_i - s_{\min})$ be the total surplus available. Each subnetwork j with $s_j \geq s_{\min}$ has its share reduced by:

$$s_j := s_j - D \cdot \frac{s_j - s_{\min}}{S} \tag{27.}$$

This redistribution preserves the relative ordering of shares above the minimum while guaranteeing all subnetworks receive sufficient time. The normalized shares still sum to 1.0 and are used throughout both solving phases.

The complexity-weighted approach offers two advantages. First, it allows harder subproblems to utilize more computational time, potentially improving solution quality where it matters most. Second, it prevents easy subproblems from consuming time they do not need.

However, complexity-based allocation has limitations. The heuristic complexity score may not accurately predict actual solver runtime, which depends on many more factors than variable counts alone. Furthermore, if the complexity estimate underallocates time to a critical subproblem, that subproblem may terminate prematurely with a poor or no solution, degrading the quality of the overall decomposed solution or making it impossible to solve. Improvements and future possibilities are discussed in Section 9.

6 Implementation

This chapter describes the technical implementation of the decomposition framework and its architecture.

6.1 System Architecture

The system is realized as an external orchestration layer written in Python that operates independently of the SAP IBP Supply Planning optimization engine while leveraging its solver infrastructure through a service interface. The framework coordinates multiple solver invocations without modifying the underlying optimization engine. This architectural decision stems from the requirement to preserve the correctness and modularity of the production solver while enabling experimental decomposition strategies. Embedding decomposition logic directly into the optimization engine would require extensive modifications to a complex codebase and would complicate the process for the *Proof-of-Concept (PoC)* of this thesis.

Figure 5 loosely illustrates the overall system architecture, showing the interaction of local modules and the remote optimization service.

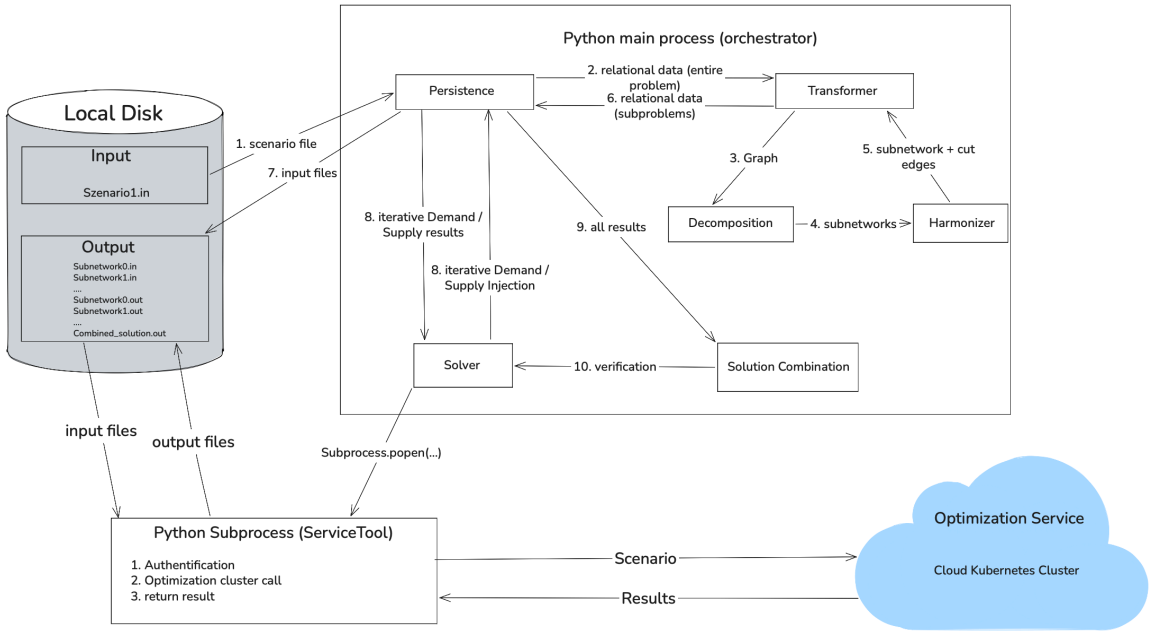


Figure 5: System architecture of the decomposition framework showing the orchestration layer and solver service interaction

6.1.1 Module Overview

The framework is organized into the following packages:

- **domain**: Contains domain models, data mappings and configuration for supply chain entities. Defines node type enumerations, node and edge resolution from database tables and heuristic configuration parameters.
- **algorithms**: Implements the core graph algorithms. The `transformer` module handles bidirectional conversion between database tables and graph representation (Section 5.1). The `decomposition` subpackage contains `static_decomposer` for stage classification and graph partitioning (Section 5.2) and `subnetwork_harmonizer` for cut edge handling and meta-graph construction. The `solver` subpackage implements `static_solver` for the two-phase solving approach and `executor` for invoking the Optimization Service (Section 5.3), `solution_harmonizer` for combining subnetwork solutions (Section 5.6) and `constraint_propagation` for cross-boundary constraint handling. The `initial_heuristics` subpackage provides `aggregation_propagation` for computing link node parameters (Section 5.4).
- **persistence**: Handles file I/O. Includes `file_parser` and `file_writer` for database serialization.
- **orchestration**: Coordinates the overall workflow through the `orchestrator` module, which sequences the transformation, decomposition, solving and solution combination phases as described in Section 5.3.

6.1.2 ServiceTool Integration

The framework communicates with the SAP *IBP* optimization service through a command-line interface tool called ServiceTool. This tool acts as a bridge between the local Python environment and the cloud-based optimization infrastructure. ServiceTool performs OAuth2 authentication against the service cluster, then uploads the input file along with the solver configuration. After the optimization completes on the remote infrastructure, ServiceTool downloads the solution archive containing the result tables. The framework invokes ServiceTool as a subprocess from Python using the `subprocess.Popen` interface with streaming output capture.

6.1.3 Error Handling and Recovery

The executor module implements retry logic with exponential backoff for transient failures such as network timeouts or temporary service unavailability. Permanent failures, such as solver time limit reached or infeasible problems, are detected through exit code analysis in the solver output and raise dedicated exceptions that terminate the current solve attempt without retry.

The framework architecture supports recovery functionality through checkpointing of intermediate results. Each subnetwork solution is persisted to disk immediately after completion, which enables recovery from failure after every fully finished optimization run. The system can detect which subnetwork was the last one completely solved and continue from there.

6.2 Data Handling and Technologies

The framework operates on data stored as files on the local filesystem. This file-based approach enables inspection of intermediate subnetwork solutions and provides natural checkpoints for the solving process.

Python was selected as the implementation language for several reasons [39]. The ecosystem provides libraries for graph algorithms and large data operations. The language's flexibility supports rapid prototyping and fast development of decomposition approaches. Furthermore, Python's subprocess management enables integration with the ServiceTool command-line interface.

The framework employs the following primary libraries:

- **NetworkX** [40]: Used for graph representation and algorithms. The supply chain network is modeled as a directed graph using NetworkX's `DiGraph` representation. The library contains several graph algorithms like traversals or condensation which the framework uses throughout.
- **Polars** [41]: Used for tabular data manipulation. Polars provides high-performance DataFrame operations for filtering and concatenating the master data tables and solver result tables. The library is used throughout SAP Supply Planning scripts and has direct integration with the table data format.
- **NumPy** [42]: Used for numerical computations on graph node and edge attributes, particularly for the aggregation heuristics.

After partitioning the supply chain graph into subgraphs as described in Section 5.2, each subnetwork must be converted into a standalone scenario file that the solver can process independently. The framework achieves this through a table filtering mechanism that extracts the data rows relevant to each subnetwork from the entire monolithic file. Each table in the data schema defines retain-if rules that specify which graph node types a row in the table references. The `columns` mapping specifies which table columns must match the corresponding node attributes for a row to be retained. Listing 1 shows an example of such a rule for the production source item table, which stores the bill of materials for production rules. This table references both the production node through location and source and the component's location-product node. Multiple retain-if items in the list are AND-concatenated, meaning a row is only retained if it matches nodes from all specified types in the subgraph.

```
RetainIfList(
    items=[
        RetainIf(
            node_type=NodeType.PRODUCTION,
            columns={"LOCID": "LOCID", "SOURCEID": "SOURCEID"}
        ),
        RetainIf(
            node_type=NodeType.LOCATION_PRODUCT,
            columns={"COMPONENTPRDID": "PRDID", "LOCID": "LOCID"}
        ),
    ],
    mode=RetainIfMode.AND
)
```

Listing 1: Retain-if configuration for the production source item table

When generating the scenario file for subnetwork i, the framework collects all node identifiers and attributes present in subgraph G_i, then filters each table to retain only rows whose key columns reference nodes within that subgraph. The filtered tables are then serialized into a binary scenario file that represents a standalone optimization scenario.

This pipeline ensures that each subproblem receives exactly the data required to optimize its portion of the supply chain, with cut edges handled through the virtual procurement mechanism described in Section 5.2.

The underlying optimization engine uses Gurobi as its *MILP* solver. Gurobi is a commercial solver known for its performance on large-scale mixed-integer programs [12]. SAP Supply Planning uses Gurobi for most optimization runs due to its good performance. Therefore, the framework uses it here as well. The complete solver parameter configuration is documented in Appendix 3.

The optimization problems are solved on the SAP Performance Optimization Cluster, a cloud-based infrastructure. Although the actual hardware can differ based on the assigned computational node, experiments showed that the differences between different runs are negligible compared to the runtimes. All experiments in this thesis, both monolithic and decomposed solves, were executed on this same infrastructure to ensure fair comparison.

6.3 Solution Verification

The decomposition framework produces solutions through a sequence of coordinated subproblem solves and combines them. The framework verifies the combined solution against the original monolithic formulation to confirm that the decomposed solution is feasible for the original problem and to obtain the true objective function value under the complete cost structure.

The verification procedure takes the original monolithic problem and adds the values from the decomposed solution as pseudo hard constraints. Specifically, the production quantities VO, transport quantities VT and procurement quantities VP from the decomposed solution are injected into the problem formulation.

Pseudo hard constraints add a penalty term to the objective function for any deviation from the target value. For each decision variable VX_i with decomposed solution value VX_i^*, the solver adds:

$$M \cdot |\mathrm{VX}_i - \mathrm{VX}_i^*| \tag{28.}$$

to the objective function where M is a large penalty coefficient, defined by $M = 10 \cdot C$ where C is the highest cost in the model. This soft constraint approach allows the solver to deviate from the injected values if necessary to maintain feasibility, while strongly preferring the original solution values. In practice, the penalty is large enough that the solver converges to the adjusted values unless doing so would violate hard constraints, because the verification runs are solved to optimality. The verification uses pseudo hard constraints rather than hard constraints because the decomposition may produce

solutions with minor numerical deviations. These deviations are tolerated by the soft constraint approach and then corrected by the verification. To differentiate between infeasible solutions and numerical issues, the decomposed solution is considered valid if no pseudo hard constraint is violated by more than 10^{-4}.

The verification solve completes rapidly compared to the original optimization. With the primary decision variables fixed, the remaining problem reduces to computing the implied values of dependent variables and evaluating the objective function. The solver does not need to explore the combinatorial space of production and transport decisions, as these are predetermined by the injected constraints. In practice, no verification run exceeded a runtime of 1% of the original runtime. This efficiency enables verification to be performed after each decomposed solve to obtain the correct objective function value. The correct objective function value cannot be obtained directly from the decomposed solutions, because, during the two-phase solve, each subproblem optimizes a modified objective that includes virtual costs and penalties for link nodes. These virtual terms guide the decomposition but do not correspond to actual costs in the original formulation. The verification problem uses the original objective function without any virtual terms. The resulting objective value represents the true cost of the decomposed solution as it would be evaluated by the monolithic solver to allow comparison.

7 Evaluation

This chapter presents the experimental evaluation of the proposed decomposition approach. The evaluation systematically compares solutions obtained through decomposition against monolithic solving across the 13 benchmark scenarios introduced in Section 4.2.2, using the metrics defined in Section 4.2.1. Empirical observations and correlations identified during the evaluation are labeled as *Findings* and numbered sequentially. Each *Finding* is then analyzed and possible reasons for it are discussed in Section 8. Each scenario is associated with a runtime budget that reflects the time limit used by the customer in production. Table 5 shows the runtime budgets for all benchmark scenarios. The monolithic run and the decomposed run both receive the same runtime budget. If one approach solves the scenario to its proven optimality, the budget is reduced to the used time.

Scenario	**Runtime Budget (s)**	**Runtime Budget (h)**
S1	31000	8.6
S2	21600	6.0
S3	1800	0.5
S4	1600	0.4
S5	10800	3.0
S6	14800	4.1
S7	20000	5.5
S8	18000	5.0
S9	5	< 0.01
S10	114	< 0.1
S11	18800	5.2
S12	45000	12.5
S13	1800	0.5

Table 5: Runtime budgets for benchmark scenarios

Table 6 presents the objective function values and optimality gaps for both approaches. The **Mono. Gap** and **Decomp. Gap** columns show how far each solution deviates from the monolithic bound. The **Avg. Subproblem Gap** column shows the average of how far each subproblem deviates from its own bound. The **Relative Diff.** is calculated as $\left(\frac{\text{Decomposition} - \text{Monolithic}}{\text{Monolithic}}\right)$, showing the percentage difference between the

two approaches, where lower objective values are better and thus negative percentages indicate better decomposition solutions. The temporal resolution of the runtime limit permits an accuracy of only two significant figures, because the solver cannot terminate instantly when reaching the runtime limit. Therefore, objective function values in Table 6 are rounded to two significant figures. Runs that did not provide a valid solution are reported with *inf*.

Scenario	**Monolithic**	**Mono. Gap**	**Decomposition**	**Decomp. Gap**	**Avg. Subproblem Gap**	**Relative Diff.**
S1	3.5e12	0.00	3.6e12	0.01	0.00	0.01
S2	1.1e12	0.00	1.2e12	0.01	0.00	0.01
S3	4.2e14	0.28	3.0e14	0.01	0.00	−0.29
S4	inf	inf	2.0e9	unknown	0.00	inf(Mono)
S5	1.9e17	0.36	2.0e17	0.40	0.13	0.05
S6	6.3e16	0.00	inf	inf	inf	inf(Deco)
S7	7.5e12	0.68	2.7e12	0.10	0.35	−0.64
S8	1.1e13	0.85	inf	inf	inf	inf(Deco)
S9	1.5e12	0.33	1.1e12	0.07	0.00	−0.26
S10	2.2e12	0.55	1.1e12	0.07	0.00	−0.52
S11	5.1e13	0.94	2.6e13	0.88	0.19	−0.49
S12	2.2e13	0.93	1.8e12	0.12	0.01	−0.92
S13	8.9e11	0.00	1.6e12	0.78	0.00	0.78

Table 6: Objective function values and optimality gaps for monolithic and decomposition approaches

Across the 10 scenarios for which both approaches produced feasible solutions, the average relative difference was −23%, corresponding to an average relative objective value improvement of 23% for decomposition. Of these 10 directly comparable scenarios, 6 showed improvement under decomposition while 4 showed worse results. Additionally, one scenario produced a feasible solution only under decomposition, and two scenarios produced feasible solutions only under monolithic solving.

The first notable correlation is that scenarios S1, S9 and S10 report an Avg. Subproblem Gap of 0.00 but fail to find the optimal monolithic solution, shown by the Decomp. Gap

being greater than 0.00. To verify whether this gap represented a genuinely suboptimal decomposed solution, the corresponding monolithic models S9 and S10 were solved with extended runtime in another experiment run. The monolithic solver subsequently found integer-feasible solutions matching the respective bounds implied by the decomposition gaps. This confirms that the bounds were attainable and that the decomposed solutions were not globally optimal, despite all subproblems having been solved to optimality. This leads to *Finding* 1:

> **Finding 1:** Solving all subproblems to optimality does not guarantee that the decomposition finds the global optimum.

Decomposition performs substantially better on scenarios where the monolithic solution reports a large optimality gap due to problem size. Examples include scenarios S11 (0.94 Mono. Gap and −0.49 improvement) and S12 (0.93 Mono. Gap and −0.92 improvement). This leads to *Finding* 2:

> **Finding 2:** Decomposition provides a good improvement on big scenarios that are solved far from optimality by the monolithic run.

A notable case is scenario S4, where the monolithic formulation failed to find any feasible solution within the allocated runtime budget, while decomposition successfully produced a solution with an objective value of 2.0e9. Since the monolithic run provides neither a solution nor a bound, the quality of the decomposition solution cannot be directly assessed through gap comparison. However, two indicators suggest the solution is reasonably good: first, all subproblems were solved to optimality and second, the customer demand fulfillment rate reports 95%. These characteristics, combined with the observation that decomposition at least provides a usable solution where monolithic solving provides none, demonstrate a practical advantage of the decomposition approach for scenarios that exceed the monolithic solver's capabilities within given time constraints. This leads to *Finding* 3:

> **Finding 3:** Decomposition can provide feasible solutions for scenarios where monolithic solving fails entirely within the given time budget.

Contrary to *Finding* 2, scenarios S9 and S10 are small scenarios that report notable improvements despite consisting of few variables and constraints. Both scenarios share

the same master data, but the discrete planning horizon of S10 is four times larger than S9, resulting in more discrete variables and a harder problem. This explains why the improvement of S10 exceeds that of S9, showing that the benefit of decomposition can increase with scenario size when the underlying network structure remains the same. This strongly reinforces *Finding* 2. The general reason for the improvement by decomposition on these two scenarios despite their small size becomes apparent when examining the virtual demand delivery results. Using S9 as a representative example for both scenarios, the virtual demand delivery rate across all decomposed runs reaches 98.5%, which explains why decomposition almost finds the optimal solution with a gap of only 0.07. Additionally, the subproblems show an evenly distributed problem severity, with problem sizes reported in Table 7.

Subnetwork	**Variables**	**Discrete Variables**	**Constraints**
bottom_up_0	17962	2876	8262
bottom_up_1	13108	1630	6699
bottom_up_2	14821	1872	6232
top_down_1	18802	1636	6330
top_down_2	14306	1156	7488

Table 7: Subproblem sizes for scenario S9

The high virtual demand delivery rate of 98.5% and the evenly distributed problem sizes across subnetworks may stem from the same root cause. The supply chain network shows a clear hierarchical organization that naturally aligns with the chosen decomposition into procurement, production and distribution stages. This manifests in the network topology through a uniform depth from procurement to customer and a clean separation of delivery decisions. Different possibilities exist to produce or transport a product, but those decisions remain within one subnetwork and both alternatives lead to the same link nodes, enabling all subnetworks to make optimal decisions with little optimality lost. When such a stage-aligned network is partitioned along its natural stage boundaries, the resulting subproblems inherit balanced complexity. The network topology visualization in Figure 6 illustrates this structure further, showing the clear separation between supply chain stages. Characteristics that would oppose this stage-aligned pattern include cycles, customer products at different depths of the graph, especially at production or procurement level, sibling nodes that are both production and customer nodes within the same subnetwork, varying numbers of production steps per product and operations spanning nodes at different stages of the supply chain.

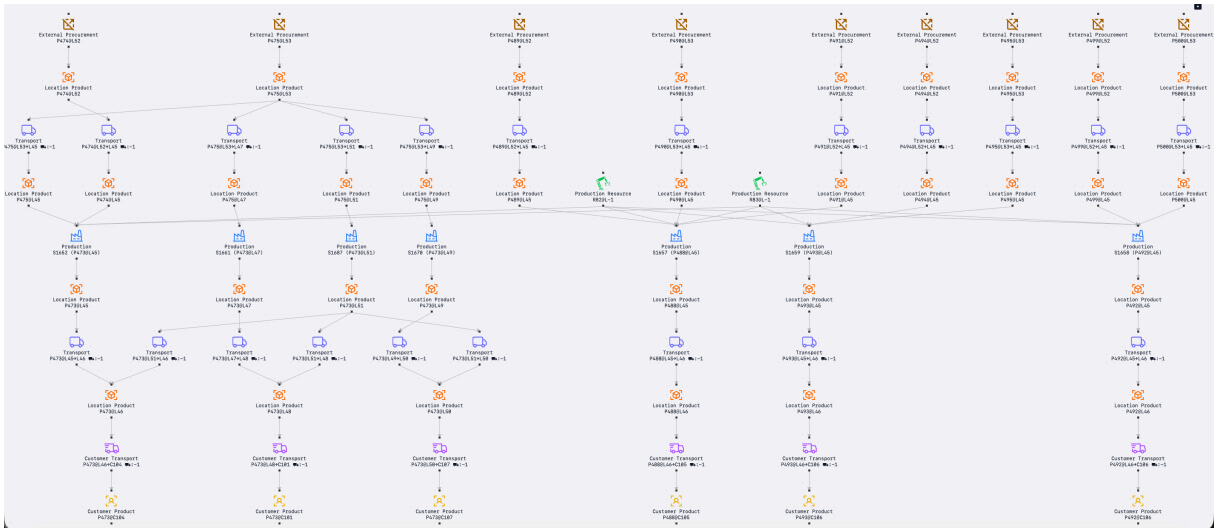


Figure 6: Network visualization showing regular structure and clear stage separation

This analysis about scenarios S9 and S10 is formulated as *Finding* 4:

> **Finding 4:** A "stage-aligned" supply chain leads to high virtual demand fulfillment rates, evenly distributed problem severity and therefore good decomposition results.

Scenario S6 reports no feasible decomposition solution, while the monolithic run converged to optimality. The decomposition aborted because the top-down production subnetwork found no solution within its allocated time limit of 12,500 seconds. Notably, the same subnetwork solved to optimality in only 109 seconds during the bottom-up phase. The only differences between the two formulations are the removal of virtual procurement variables and the injection of fixed supply quantities from the upstream subnetwork into the inventory balance constraints. To investigate this discrepancy, Table 8 compares the model statistics of both formulations. Despite the top-down model having fewer variables and constraints, it fails to find any solution while the bottom-up run reaches optimality.

Metric	**Bottom-up**	**Top-down**
Variables	149,699	119,859
Discrete Variables	7,048	6,803
Constraints	74,699	70,502
Solve Time (s)	109	12,548
Solution	Optimal	None

Table 8: Model comparison of S6 production subnetwork (bottom-up vs. top-down)

This observation is highly unintuitive: the solve time is 115 times higher on essentially the same problem with slightly fewer variables, yet the solver found no solution. To understand this discrepancy, Table 9 compares the internal solver behavior between both runs.

Metric	**Bottom-up**	**Top-down**
Root LP Time (s)	1.44	440.69
Crossover Time (s)	—	271.24
First B&B Node (s)	43	2,110
Total B&B Nodes	114,706	15
Final Result	Near-optimal (0.01% gap)	No solution

Table 9: Solver behavior comparison for S6 production subnetwork

The bottom-up run solves the root LP relaxation quickly and explores over 100,000 branch-and-bound nodes. In contrast, the top-down run spends significant time on the root LP relaxation alone, requiring an additional Crossover phase of 271 seconds. The first branch-and-bound node takes 50 times longer than in the bottom-up run and within the remaining time budget, the solver explores only 15 nodes without finding any integer-feasible solution.

Investigation of S8, the other scenario where decomposition failed, reveals the same pattern. The top-down subnetwork found no solution within 7,900 seconds, while the bottom-up run of the same subnetwork reached near-optimality within 3,200 seconds. This behavior generalizes across many scenarios. Table 10 shows that top-down subproblems consistently require more time and achieve worse gaps than their bottom-up counterparts.

Metric	**S2**	**S11**	**S12**
Bottom-up Time (s)	1,900	7,524	9,000
Top-down Time (s)	9,100	11,240	9,000
Bottom-up Gap	0.00	0.08	0.00
Top-down Gap	0.00	0.88	0.08

Table 10: Top-down vs. bottom-up performance across scenarios

To identify the root cause of this performance difference, the LP relaxation of S8′s top-down production subnetwork was analyzed in isolation. Table 11 shows the progression through solver phases.

#	**Phase**	**Time (s)**	**Observation**
1	Barrier	27	Converged normally (64 iterations)
2	Crossover	969	Aborted due to numerical instability
3	Dual Simplex	5,400	Objective function value constant ($\approx$ 1.59e12), infeasibility oscillations (10^7–10^{18}), Markowitz Tolerance warnings
—	**Total**	**6,400**	Bottom-up LP Comparison: 7 seconds

Table 11: LP relaxation solver phases for S8 top-down production subnetwork

The Barrier method converges quickly to an interior-point solution, but the subsequent Crossover phase, which converts this solution into a basic feasible solution, fails due to numerical instability. The solver then falls back to dual simplex, where it spends 5,400 seconds with effectively no progress on the objective function while infeasibility residuals oscillate across 11 orders of magnitude.

These observations are classical symptoms of problem degeneracy [2], [10], [11]:

- Rapid Barrier convergence to an interior-point solution
- Long Crossover phase turning the interior-point solution into a basic solution
- Infeasibility oscillations spanning 10^7 to 10^{18}
- Markowitz tolerance warnings [43]
- Minimal objective improvement during simplex iterations

None of these symptoms appeared in the bottom-up run. To verify the diagnosis of problem degeneracy, an additional experiment run of the LP relaxation of S8′s top-down production subnetwork forced the solver to use dual simplex directly, bypassing Barrier and Crossover entirely. The LP then solved in 772 seconds, which is still elevated compared to the bottom-up run, because dual simplex is also affected by the underlying degeneracy, but faster than the run using Barrier and Crossover. This supports the diagnosis that the top-down transformation introduces or substantially increases degeneracy in the formulation. When the LP relaxation exhibits this degenerate behavior, the relaxations at branch-and-bound nodes are likely to exhibit it as well, thereby slowing down the entire solving process.

To summarize, the following finding is reported, with reasons identified and improvements discussed in Section 8:

> **Finding 5:** Top-down subproblems exhibit severe degeneracy, causing them to require substantially more time than their bottom-up counterparts and becoming a runtime bottleneck that degrades overall decomposition performance.

Following the general analysis and identification of the root cause for infeasible scenarios, the next step examines why scenarios, that did not show improvement, performed poorly. Scenarios that report a relative difference of Relative Diff. > 0.00 include S1, S2, S5 and S13.

A common pattern emerges across all scenarios that perform worse under decomposition: an uneven distribution of problem complexity across subnetworks, with the production subnetwork (subnetwork 1) consistently being the hardest. Table 12 shows the estimated complexity distribution across subnetworks.

Scenario	**Subnetwork 0 (%)**	**Subnetwork 1 (%)**	**Subnetwork 2 (%)**
S1	10	80	10
S2	0 (empty)	90	10
S5	10	80	10
S13	0 (empty)	86	14

Table 12: Complexity distribution across subnetworks for scenarios with worse decomposition results

This observation directly relates to *Finding* 4, but represents its inverse. While stage-aligned supply chains with balanced subproblem complexity lead to good decomposition results, supply chains with a dominant production stage lead to poor results. This characteristic peaks in so-called "production scenarios". Such scenarios include S13, where the Procurement stage is very small or non-existent because the scenario was modeled with infinite inputs for all productions. This modeling choice reflects situations where the company has virtually unlimited access to raw materials and primarily needs to determine production quantities. These scenarios can be identified by their low number of Supplier or Procurement nodes, as shown in Table 1 where S13 reports only 6 Procurements. This uneven complexity arises not only from the sheer number of nodes in the production stage, but also from the concentration of discrete lot sizes in production operations. For instance, S5 contains approximately three times as many discrete production variables as discrete transport variables, because transportation decisions are often modeled as continuous while production lot-sizing decisions require

discretization. When decomposition faces such an unbalanced problem structure with one dominant subproblem, the decomposition approach cannot leverage its advantage effectively.

> **Finding 6:** Decomposition performs poorly when the production subnetwork dominates problem complexity due to a concentration of discrete decision variables.

The most critical structural limitation of the approach becomes apparent when analyzing the specific results of scenarios where decomposition produced worse overall results than monolithic solving. This observation identifies the root cause of *Finding* 1, which showed that decomposition cannot find the global optimum even when all subproblems are solved to optimality. Table 13 shows that decomposition performed worse because of lower customer demand fulfillment.

Scenario	**Customer Demand Mono. (%)**	**Customer Demand Decomp. (%)**	**Virtual Demand Fulfillment (%)**
S1	88	87	90
S2	92	91	95
S5	39	37	52
S13	96	82	84

Table 13: Demand fulfillment rates for scenarios with worse decomposition results

Examining customer demands that were fulfilled in the monolithic run but not during decomposition reveals a consistent pattern. These demands trace back to unfulfilled virtual demands due to suboptimal request placement. Either the bottom-up phase requested material in periods where upstream capacity was already exhausted while earlier periods still had availability, or requests were routed through link nodes whose upstream paths faced bottlenecks while alternative link nodes could have delivered.

This represents a fundamental structural limitation of the two-phase decomposition. The bottom-up phase commits to specific inter-stage flows without knowledge of whether those flows can actually be fulfilled by upstream stages and the optimal procurement paths may require requesting from additional link nodes. Since the single up-and-down pass cannot revisit these decisions after the correction phase fails to deliver, material that could have been procured through alternative routes remains unutilized. This behavior is most pronounced in scenarios that provide multiple paths from different procurements or productions to the same output customer product,

meaning the decision of which path to choose must be made in the lowest subnetwork without access to information about capacity limitations in higher subnetworks. Since resource capacity limits represent the most frequent capacity limitations and therefore the most frequent reason for unfulfilled virtual demands, this behavior intensifies when single resources are shared across many operations, resulting in a high number of resource consumers per single resource. Scenario S5 exemplifies this pattern with only 175 production resources but 106,394 production resource consumers.

The following finding is reported, with two possible improvements proposed in Section 9:

> **Finding 7:** Decomposition can yield worse solutions than monolithic solving, especially on scenarios with shared resources or multiple production or transport alternatives.

8 Discussion

This chapter interprets the evaluation findings on a conceptual level, discusses their implications for practical application of stage decomposition and identifies methodological limitations.

The evaluation demonstrates that stage decomposition can substantially improve solution quality on large, complex scenarios where monolithic solving struggles. However, the approach exhibits fundamental structural limitations that stem from its two-phase design and underlying nature of decomposition.

8.1 Interpretation of Findings

Finding 1 states that decomposed runs fail to find the global optimum even when all subproblems are solved to optimality. *Finding* 7 identifies the root cause as suboptimal inter-subnetwork flow commitments during the bottom-up phase. Together, these findings point to a fundamental information asymmetry problem inherent in the hierarchical decomposition approach because of binding decisions by the bottom-up about material flows without visibility into upstream capacity constraints. The underlying issue is that stage decomposition breaks the global coordination that monolithic solving provides. Classical decomposition methods like Dantzig-Wolfe [5] and Benders decomposition [16] address this through iterative information exchange between master and subproblems. Lagrangian relaxation [17] uses iterative price signals to coordinate decoupled subproblems. The stage decomposition trades this iterative coordination for computational simplicity and speed, accepting that some global optimality will be lost. Two possible improvements to mitigate this limitation are discussed in Section 9.1.

Finding 2 states that decomposition provides good improvement on large scenarios solved far from optimality by monolithic runs. This observation aligns with the superlinear scaling behavior of mixed-integer programming solvers [2]. As problem size increases, monolithic solve time grows faster than linearly due to the combinatorial explosion in the branch-and-bound tree. Decomposition counteracts this by partitioning the problem into smaller subproblems whose combined solve times remain smaller than the monolithic solve time. The practical implication is that decomposition should primarily be applied to scenarios that are too large for monolithic solving to reach good solutions within the available time budget. For smaller scenarios that can be solved optimally by monolithic approaches, the potential optimality loss might not be justified.

However, *Finding* 4 identifies an alternative justification for using the approach also on selective small scenarios.

Finding 4 identifies that stage-aligned supply chains can also enable solution improvements by decomposition, because it produces similar sized problems and reduces the information loss at subnetwork boundaries. The practical implication is that decomposition should be used for scenarios that exhibit this favorable topology.

Finding 3 shows that decomposition can provide usable solutions in cases where the monolithic solver fails to find any feasible solution within the given time budget. Notably, decomposition obtains a solution that is optimal with respect to the individual decomposed subproblems, while the monolithic approach finds no solution at all. The decomposition decouples constraints that prevent the monolithic solver from making progress. This introduces a practical use case for decomposition as a fallback strategy. When monolithic solving fails to produce any result within the allocated time, enabling decomposition offers an alternative path to obtaining a solution rather than requiring an increase in the runtime budget.

Finding 6 conversely shows that production-dominated scenarios perform poorly under decomposition. This is expected because decomposition benefits diminish when one subproblem dominates the overall complexity. The practical implication is that decomposition should only be applied when problem complexity is reasonably distributed across stages. Scenarios where transportation decisions are also modeled as discrete variables tend to exhibit more balanced complexity. Conversely, production scenarios where procurement is trivial and distribution is continuous should avoid this decomposition approach entirely.

Finding 5 identifies that top-down runs consistently require more time than bottom-up runs despite similar complexity metrics, likely due to problem degeneracy. This finding reveals an unintended consequence of the virtual demand mechanism. During bottom-up, virtual demand variables act as slack variables that provide flexibility in satisfying link node inventory balance constraints, because these variables are not part of any other constraint and can be chosen freely. Their removal in top-down creates a more constrained system where balance must be achieved through tightly coupled production and transportation variables. The mathematical interpretation is that the feasible polyhedron becomes more degenerate when independent slack variables are replaced with dependent variables participating in multiple constraints. Degenerate linear programs are known to cause numerical difficulties for simplex-based methods

and Crossover [10], explaining the observed solver behavior. A potential improvement using Dual Simplex warmstarting is discussed in Section 9.2.

To sum up, decomposition reaches its best achievable solution faster than monolithic solving reaches the same quality level, unless it encounters the degeneracy problems observed in several scenarios. Moreover, in scenarios where subproblems were not yet solved to optimality within the time budget, decomposition consistently outperformed monolithic solving, demonstrating that the approach better utilizes the same limited computation time. However, decomposition cannot improve beyond this structural optimum imposed by the information asymmetry at stage boundaries. Monolithic solving can eventually find better solutions given extra time. Based on these observations, stage decomposition should primarily be applied to large scenarios and/or stage-aligned supply chains with balanced complexity across stages. Decomposition should be avoided for production-dominated scenarios where discrete variables concentrate in production while procurement and distribution are comparatively simple, as well as scenarios with high resource sharing.

8.2 Solution Quality Assessment

The evaluation reveals a fundamental problem regarding solution quality assessment. Decomposition provides no global optimality bound because the bounds from individual subproblems do not compose into a valid bound for the original monolithic problem. Therefore, practitioners cannot determine the optimality gap of the decomposed solution.

However, this could also impose an advantage, since a large portion of monolithic solve time typically goes toward proving optimality rather than finding good solutions [9]. Decomposition mitigates this by requiring optimality proofs only for individual subproblems rather than the full problem. Given the non-linear scaling of optimality proofs, proving smaller subproblems optimal requires less time than proving the monolithic problem optimal.

8.3 Limitations

Several implementation limitations affect the scope and generalizability of this work.

The stage decomposition approach introduces computational overhead for graph analysis, subproblem construction, coordination logic and result assembly. For the evaluation, time used for these computations was not reported. For large scenarios where solve time dominates total runtime, this overhead is negligible. For smaller scenarios that solve quickly monolithically, the overhead can exceed the solve time benefits. The evaluation did not systematically quantify this overhead or identify the problem size threshold where decomposition becomes beneficial, as discussed and justified in Section 4.

Certain constraint types in the SAP IBP optimization model cannot be decomposed within the current framework and scenarios containing those were excluded from the analysis. Late delivery requires global information to determine which link node demands will ultimately be mapped to which customer demands. Initial inventory of materials prevents the application of dead-end detection, lead time aggregation and marginal cost heuristics, because they rely on proportional relations. Minimum aggregated constraints cannot be partitioned because the behavior of other subnetworks cannot be predicted at decomposition time and therefore the constraint cannot be decomposed meaningfully. Because of the listed reasons, scenarios using these features are not supported.

The evaluation focused exclusively on stage decomposition with three fixed stage groups corresponding to procurement, production and distribution. Alternative decomposition strategies exist that partition along different dimensions, for example, product decomposition. Product decomposition creates subproblems per product family. The evaluation does not compare stage decomposition against product decomposition, leaving open whether stage boundaries represent the most effective partition strategy for supply chain optimization. Furthermore, hybrid approaches combining stage decomposition with product decomposition could potentially achieve better results than any single decomposition dimension.

The benchmark scenarios, while drawn from real customer data at SAP, represent a convenience sample rather than a representative sample of all supply chain optimization problems. The scenarios vary in size and structure but share common characteristics imposed by the SAP IBP modeling framework. Decomposition performance on supply chains modeled in different frameworks or with fundamentally different structures may differ from the observed results.

9 Future Work

This chapter discusses potential directions for future research building on the presented decomposition framework. The following section outlines improvements to address identified limitations and extensions that could broaden the applicability of the approach.

9.1 Improvement of Information Asymmetry

Two possible improvements to the design could mitigate the information asymmetry limitation identified in *Finding* 1 and *Finding* 7.

9.1.1 Resource Capacity Aggregation

This approach propagates approximate resource capacity information from upstream subnetworks down to the link nodes before the bottom-up run. For each combination of resource r and link node l, the approach computes a coefficient $c_{r,l,T}$ representing the maximum amount of resource r consumed per unit of link node product delivered up to period T. These coefficients then form aggregated constraints on the link nodes that approximate the upstream resource limitations. For each period T, the constraint would ensure that the cumulative resource consumption implied by the virtual demand requests does not exceed the cumulative resource capacity up to period T:

$$\sum_{t=0}^{T} \text{Capa}_{r(t)} \geq \sum_{t=0}^{T} \sum_{l \in L_r} c_{r,l}(t) \cdot \text{VP}_l(t) \tag{29.}$$

where L_r denotes all link nodes that have resource r in their upstream subnetwork and $\text{VP}_l(t)$ is the virtual demand quantity requested at link node l in period t. By incorporating these aggregated resource constraints into the bottom-up formulation, the subnetwork gains visibility into upstream capacity limitations before committing to specific virtual demand quantities, potentially increasing the virtual demand fulfillment rate.

9.1.2 Iterative Extension

Iterative refinement can further improve decomposition solutions by progressively shrinking the problem and re-solving. After the first bottom-up and top-down pass, much of the network is already planned, parts of the supply chain are committed and usually only few customer demands are unfulfilled. The key insight is that the settled

portions of the supply network can be removed from the problem entirely, leaving a much smaller network to re-solve. Two categories of elements are removed after each iteration:

1. Fulfilled customer demands: When a customer demand is completely satisfied, its entire upstream supply path can be removed, provided those operations do not serve other unfulfilled demands. These decisions are final and need no further optimization.
2. Capacity-limited paths: When a virtual demand is only partially fulfilled, the procurement path hit a capacity constraint. The segments upstream and downstream of this bottleneck, up to the next branching point, can be removed, since they cannot fulfill additional demands, or they would have already.

The quantities produced along the segments are fixed and subtracted from resource or storage capacities. The result is a substantially reduced problem. Re-solving this smaller network allows the decomposition to explore alternative procurement routes that were bypassed in the first pass. Since the fixed quantities represent feasible partial solutions, feasibility is preserved when combining iterations. This process can be repeated until virtual demand fulfillment reaches 100% or no further improvement occurs. Because the first pass already satisfies most demands, subsequent iterations operate on progressively smaller problems and should solve quickly.

9.2 Improvement of Top-Down Problem Degeneracy

Possible improvements could mitigate the degeneracy of the top-down problems identified in *Finding* 5.

Using Dual Simplex for the root relaxation and for the relaxations on branch-and-bound nodes could improve performance by avoiding the expensive Crossover phase.

A massive speedup of top-down runs could be achieved by using Dual Simplex with a warm start from the bottom-up solution. For portions of the problem where virtual demands were satisfiable, the top-down solution will closely match the already known bottom-up result. The top-down run then only needs to resolve the portions where virtual demands could not be fulfilled, because there, the bottom-up solution is infeasible.

An alternative application of the decomposition approach emerges from the finding that decomposition can find good solutions faster than monolithic solving, but fails to converge to optimality. Rather than using decomposition as a standalone solving

strategy, it could serve as a rapid heuristic to generate high-quality starting solutions for subsequent optimization. The decomposed solution could provide a warm start for monolithic solving or serve as an initial incumbent for other metaheuristic approaches [2]. This hybrid strategy would combine the speed advantage of decomposition with the optimality guarantees of monolithic methods.

9.3 Improvement of Inter-Subnetwork Constraint Handling

Two possible alternative approaches could improve the simple resource inter-subnetwork constraint handling presented in Section 5.5.

9.3.1 First-Come, First-Served

The *FCFS* strategy allocates resource capacity dynamically based on actual usage during the top-down phase. In the bottom-up phase, all subnetworks receive access to the full original capacity. In the top-down phase, the first subnetwork retains its bottom-up solution unchanged, effectively using full capacity, and its actual consumption is recorded. Each subsequent subnetwork then receives only the remaining capacity:

$$\mathrm{RC}_r^{(i)}(t) = \mathrm{RC}_{r(t)} - \sum_{j \text{ solved before } i} \sum_{o \in O_j} \mathrm{RR}_{o,r} \cdot \mathrm{VO}_o^{(j)}(t) \qquad 30.$$

This approach aligns capacity allocation with material flow direction. However, upstream subnetworks may consume all available capacity, leaving downstream subnetworks with insufficient capacity to fulfill demands.

9.3.2 Preallocation

The Preallocation strategy provides an informed capacity allocation by first solving the LP relaxation of the entire monolithic problem. The resulting fractional solution provides guidance on how capacity would be utilized under optimal continuous conditions and how it should be distributed for the *MILP*. After solving the LP-relaxation, the capacity is distributed across subnetworks proportionally based on the LP-derived usage. For a shared resource r with capacity $\mathrm{RC}_{r(t)}$, let N_r be the set of subnetwork indices containing operations that consume r. The total LP usage by operations in subnetwork i is $U_i^{\mathrm{LP}(t)} = \sum_{o \in O_i} \mathrm{RR}_{o,r} \cdot \mathrm{VO}_o^{\mathrm{LP}(t)}$. The capacity allocated to each subnetwork $i \in N_r$ is:

$$\mathrm{RC}_r^{(i)}(t) = \mathrm{RC}_{r(t)} \cdot \frac{U_i^{\mathrm{LP}(t)}}{\sum_{j \in N_r} U_j^{\mathrm{LP}(t)}} \tag{31.}$$

Equation 31 distributes the entire original capacity $\mathrm{RC}_r(t)$ across all subnetworks that have operations consuming the resource, using the LP solution only to determine the allocation ratios. That means that even if the LP solution does not use the whole available capacity, each subnetwork receives its proportional share of the full capacity. Since all proportional shares sum to 1, this ensures that unused capacity in the LP solution remains available for the integer subproblems, even if it was not used in the LP solution.

This approach offers several advantages over the alternatives. Unlike equal split, the allocation reflects actual optimization priorities rather than treating all subnetworks uniformly. Unlike *FCFS*, all subnetworks receive their allocation simultaneously based on global information, avoiding the sequential bias where solve order determines capacity access. The primary limitation is that the LP relaxation might not be trivial to solve for large scenarios and it takes up additional runtime before the actual solving can begin.

9.4 Improvement of Runtime Allocation

The current runtime allocation mechanism uses a complexity score based on variable counts to distribute the total time budget across subnetworks. However, this heuristic may not accurately predict actual solver runtime, which depends on many factors beyond variable counts alone. When the complexity estimate underallocates time to a critical subproblem, that subproblem may terminate prematurely without finding a solution, making the entire decomposition infeasible. This approach could be improved in a number of ways.

Instead of estimating complexity before model construction, the allocation could be performed by first building all subnetwork models in the beginning. The virtual demand variables cannot be included in this initial assessment since they are only known after the bottom-up solve, but they are easier to estimate compared to the base model size. The base models could be kept in memory and the virtual demands and supplies added just before each subnetwork is solved. This would enable more accurate complexity assessment based on the actual model structure rather than heuristic estimates.

The current approach terminates subproblems when their allocated time budget expires, even if no solution has been found. A more robust strategy would be to continue solving a subproblem beyond its allocated budget if it has not yet found a feasible solution, potentially reallocating the entire remaining global budget to that critical subproblem. This is justified because when any subproblem fails to find a solution, the entire decomposition run cannot produce a valid solution due to the dependencies between subnetworks. Therefore, investing all available time to obtain at least one feasible solution for the blocking subproblem increases the chance of obtaining any solution at all, rather than strictly adhering to budget allocations and guaranteeing failure.

10 Conclusion

This thesis developed and evaluated a stage-based decomposition approach for supply chain network optimization. The approach partitions large *MILP* problems along supply chain stages procurement, production and distribution and solves them through a two-phase bottom-up/top-down procedure that coordinates subproblem solutions via virtual demands at stage boundaries.

RQ1: How can supply chain networks be decomposed into subnetworks while preserving solution feasibility?

An approach was proposed that decomposes supply chain networks by partitioning nodes according to their stage classification within the supply chain hierarchy. Link nodes at stage boundaries serve as coordination points where virtual demand variables enable information exchange between subnetworks. The bottom-up phase determines material requirements propagating upstream, while the top-down phase commits to actual deliveries based on upstream capacity. The correction mechanism ensures that, when all required subproblems are solved successfully, the final combined solution satisfies the original constraints. The handling for inter-subnetwork constraints supports constraints spanning multiple subnetworks. Several heuristics for information aggregation improve the solution quality. However, the evaluation revealed that this two-phase coordination could lead to suboptimal inter-stage flow commitments when bottom-up decisions are made without complete visibility into upstream capacity constraints.

RQ2: How does decomposition affect solution quality and computational performance compared to monolithic solving?

The evaluation demonstrates that decomposition provides substantial improvements for large scenarios where monolithic solving struggles to find good solutions within reasonable time budgets. On scenarios with monolithic optimality gaps exceeding 0.90, decomposition achieved improvements of up to 92%. Conversely, for scenarios that can be solved optimally by monolithic approaches, decomposition may produce worse results due to the inherent information asymmetry at stage boundaries. Stage-aligned supply chains with balanced complexity across stages benefit most from decomposition, while production-dominated scenarios where discrete variables concentrate in one stage show diminished benefits.

RQ3: Can decomposition enable solving previously intractable problems?

Yes. The evaluation identified scenarios where monolithic solving failed to find any feasible solution within the allocated time budget, while decomposition successfully produced usable results. Decomposition decouples constraints that prevent the monolithic solver from making progress, offering a practical fallback strategy when increasing the runtime budget is impractical.

This work contributes a practical decomposition approach tailored to the hierarchical structure of supply chain networks. Unlike classical methods such as Dantzig-Wolfe or Benders decomposition that require iterative coordination between master and subproblems, stage decomposition trades this iterative exchange for computational simplicity and speed. The approach aligns with the broader research direction of exploiting problem structure for scalable optimization [2], while addressing the specific characteristics of multi-echelon supply chain planning. The evaluation on real-world customer scenarios from SAP *IBP* demonstrated that the approach provides practical value for enterprise-scale supply chain optimization.

Several limitations constrain the generalizability of the findings. The evaluation covered a limited set of benchmark scenarios and certain constraint types could not be decomposed. Future work could address the information asymmetry at stage boundaries and the observed degeneracy in top-down subproblems through the discussed approaches.

The central finding of this thesis is that stage decomposition can be beneficial, but performs best under specific structural conditions. The evaluation shows that decomposition improves results more often than it degrades them, yet when supply chain complexity is unbalanced or the network structure misaligns with decomposition boundaries, the approach can perform worse than the monolithic baseline it aims to replace. Further improvements are needed and practitioners should view the decomposition approach not as a universal solution, but as a tool whose applicability depends on the structural characteristics of their specific supply chain. The evaluation provides concrete criteria for this assessment, allowing an informed decision on whether decomposition will likely be beneficial.

Bibliography

[1] S. Chopra and P. Meindl, *Supply Chain Management: Strategy, Planning and Operation*, 7th ed. Pearson, 2021.

[2] L. A. Wolsey, *Integer Programming*, 2nd ed. Wiley, 2020.

[3] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman, 1979.

[4] SAP SE, "SAP Integrated Business Planning." Accessed: Jul. 18, 2026. [Online]. Available: https://help.sap.com/docs/SAP_INTEGRATED_BUSINESS_PLANNING

[5] G. B. Dantzig and P. Wolfe, "Decomposition Principle for Linear Programs," *Operations Research*, vol. 8, no. 1, pp. 101–111, 1960.

[6] A. Schrijver, *Theory of Linear and Integer Programming*. Wiley, 1998.

[7] G. Barbarosoglu and D. Ozgur, "Hierarchical design of an integrated production and 2-echelon distribution system," *European Journal of Operational Research*, vol. 118, no. 3, pp. 464–484, 1999.

[8] H. Lidestam and M. Rönnqvist, "Use of Lagrangian decomposition in supply chain planning," *Mathematical and Computer Modelling*, vol. 54, no. 9–10, pp. 2428–2442, 2011.

[9] R. E. Bixby, "A Brief History of Linear and Mixed-Integer Programming Computation," *Documenta Mathematica*, vol. Extra Volume ISMP, pp. 107–121, 2012.

[10] D. Bertsimas and J. N. Tsitsiklis, *Introduction to Linear Optimization*. Athena Scientific, 1997.

[11] E. D. Andersen and K. D. Andersen, "On Implementing a Primal-Dual Interior-Point Method for Linear Programming," *Mathematical Programming*, vol. 72, no. 3, pp. 249–277, 1996.

[12] L. Gurobi Optimization, "Gurobi Optimizer Reference Manual." Accessed: Jul. 12, 2026. [Online]. Available: https://www.gurobi.com/documentation/

[13] SCIP Optimization Suite, "SCIP: Solving Constraint Integer Programs." Accessed: Jul. 05, 2026. [Online]. Available: https://www.scipopt.org/

[14] U. Sandner, "Dekompositionsverfahren für die Optimierung der Supply Network-Planung," 1998.

[15] J. F. Shapiro, *Modeling the Supply Chain*, 2nd ed. Duxbury Press, 2006.

[16] J. F. Benders, "Partitioning Procedures for Solving Mixed-Variables Programming Problems," *Numerische Mathematik*, vol. 4, no. 1, pp. 238–252, 1962.

[17] M. L. Fisher, "The Lagrangian Relaxation Method for Solving Integer Programming Problems," *Management Science*, vol. 27, no. 1, pp. 1–18, 1981.

[18] A. M. Geoffrion, "Lagrangean Relaxation for Integer Programming," *Mathematical Programming Study*, vol. 2, pp. 82–114, 1974.

[19] H. I. Everett, "Generalized Lagrange Multiplier Method for Solving Problems of Optimum Allocation of Resources," *Operations Research*, vol. 11, no. 3, pp. 399–417, 1963.

[20] C. Lemaréchal, "Lagrangian Relaxation," in *Computational Combinatorial Optimization*, M. Jünger and D. Naddef, Eds., Springer, 2001, pp. 112–156.

[21] L. S. Lasdon, *Optimization Theory for Large Systems*. Macmillan, 1970.

[22] D. P. Bertsekas, *Nonlinear Programming*, 2nd ed. Athena Scientific, 1999.

[23] H. Stadtler, C. Kilger, and H. Meyr, *Supply Chain Management and Advanced Planning: Concepts, Models, Software and Case Studies*, 5th ed. Springer, 2015.

[24] J. T. Dickersbach, *Supply Chain Management with SAP APO: Structures, Modelling Approaches and Implementation*, 3rd ed. Springer, 2009.

[25] G. B. Dantzig, *Linear Programming and Extensions*. Princeton University Press, 1963.

[26] L. Schewe, M. Schmidt, and D. Weninger, "A Decomposition Heuristic for Mixed-Integer Supply Chain Problems," *Operations Research Letters*, vol. 48, no. 3, pp. 225–232, 2020.

[27] H.-O. Günther and H. Meyr, *Supply Chain Planning: Quantitative Decision Support and Advanced Planning Solutions*. Springer, 2009.

[28] J. R. Jackson and I. E. Grossmann, "Temporal decomposition scheme for nonlinear multisite production planning and distribution models," *Industrial and Engineering Chemistry Research*, vol. 42, no. 13, pp. 3045–3055, 2003.

[29] K. Neumann and M. Morlock, *Operations Research*. Hanser, 1993.

[30] H. P. Williams, *Model Building in Mathematical Programming*, 3rd ed. John Wiley & Sons, 1991.

[31] A. J. Clark and H. Scarf, "Optimal Policies for a Multi-Echelon Inventory Problem," *Management Science*, vol. 6, no. 4, pp. 475–490, 1960.

[32] S. A. Bessler and A. F. Veinott Jr., "Optimal Policy for a Dynamic Multi-Echelon Inventory Model," *Naval Research Logistics Quarterly*, vol. 13, no. 4, pp. 355–389, 1966.

[33] P. Glasserman and S. Tayur, "The Stability of a Capacitated, Multi-Echelon Production-Inventory System Under a Base-Stock Policy," *Operations Research*, vol. 42, no. 5, pp. 913–925, 1994.

[34] F. Chen, "Optimal Policies for Multi-Echelon Inventory Problems with Batch Ordering," *Operations Research*, vol. 48, no. 3, pp. 376–389, 2000.

[35] E. B. Diks and A. G. de Kok, "Optimal Control of a Divergent Multi-Echelon Inventory System," *European Journal of Operational Research*, vol. 111, no. 1, pp. 75–97, 1998.

[36] K. Dogan and M. Goetschalckx, "A primal decomposition method for the integrated design of multi-period production-distribution systems," *IIE Transactions*, vol. 31, no. 11, pp. 1027–1036, 1999.

[37] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[38] V. D. R. Guide Jr. and L. N. Van Wassenhove, "Closed-Loop Supply Chains: A Critical Review and Future Research," *Interfaces*, vol. 33, no. 6, pp. 57–66, 2003.

[39] Python Software Foundation, "Python Documentation." Accessed: Jul. 27, 2026. [Online]. Available: https://docs.python.org/3/

[40] NetworkX Developers, "NetworkX Documentation." Accessed: Jul. 23, 2026. [Online]. Available: https://networkx.org/documentation/stable/

[41] Polars Contributors, "Polars Documentation." Accessed: Jul. 09, 2026. [Online]. Available: https://docs.pola.rs/

[42] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[43] H. M. Markowitz, "The Elimination Form of the Inverse and Its Application to Linear Programming," *Management Science*, vol. 3, no. 3, pp. 255–269, 1957.

List of Appendices

1 Appendix: Proof: Disjointness of Stage Sets ..... B
2 Appendix: Variable-Count Estimation ..... D
3 Appendix: Solver Configuration ..... E

1 Appendix: Proof: Disjointness of Stage Sets

Premise. Let $G_C = (V_C, E_C)$ be the condensation of the supply chain graph. By construction, G_C is a *DAG*.

Definitions. Let $P = \{v \in V_C : \tau(v) = \text{Prod}\}$ be the production nodes. Define $U_P = \left\{v \in V_C : \exists p \in P, v \xrightarrow{*} p\right\}$ (nodes with production downstream) and $D_P = \left\{v \in V_C : \exists p \in P, p \xrightarrow{*} v\right\}$ (nodes with production upstream), where $\xrightarrow{*}$ denotes reachability.

Lemma 1. The sets $U_P \setminus D_P$, $U_P \cap D_P$, $D_P \setminus U_P$ and $V_C \setminus (U_P \cup D_P)$ partition V_C.

Proof. For any universe U and sets $A, B \subseteq U$, the four sets $A \setminus B$, $A \cap B$, $B \setminus A$ and $U \setminus (A \cup B)$ are pairwise disjoint and their union equals U by definition. □

Theorem. The stage sets S_1, S_2, S_3, S_4, S_5 partition V_C.

Proof. Let $R = V_C \setminus (U_P \cup D_P)$, $R_1 = \{v \in R : \tau(v) = \text{Proc} \vee \exists (u, v) \in E_C, \tau(u) = \text{Proc}\}$ and $R_5 = \{v \in R : \tau(v) = \text{Cust} \vee \exists (v, w) \in E_C, \tau(w) = \text{Cust}\}$. It is shown (i) pairwise disjointness and (ii) completeness.

(i) Disjointness. By Lemma 1, the four base regions $U_P \setminus D_P$, $U_P \cap D_P$, $D_P \setminus U_P$ and R partition V_C. Each node belongs to exactly one base region and is assigned to a stage set based only on its region. Since regions are disjoint, it suffices to show disjointness within each region.

Within $U_P \setminus D_P$: S_1 contains nodes that are procurement nodes or have procurement input, S_2 contains nodes that are neither procurement nodes nor have procurement input. These conditions are complementary, so $S_1 \cap S_2 = \emptyset$.

Within $U_P \cap D_P$: All nodes go to S_3.

Within $D_P \setminus U_P$: S_5 contains nodes that are customer nodes or have customer output, S_4 contains nodes that are neither customer nodes nor have customer output. These conditions are complementary, so $S_4 \cap S_5 = \emptyset$.

Within R: Nodes in R_1 go to S_1. Nodes in $R_5 \setminus R_1$ go to S_5. Nodes in $R \setminus R_1 \setminus R_5$ go to S_4. These three sets partition R by construction.

(ii) Completeness. By Lemma 1, every $v \in V_C$ belongs to exactly one base region:

- $U_P \setminus D_P$: Covered by $S_1 \cup S_2$ (procurement node/input or not).
- $U_P \cap D_P$: Equals S_3 by definition.
- $D_P \setminus U_P$: Covered by $S_4 \cup S_5$ (customer node/output or not).
- R: Covered by $R_1 \subset S_1$, $(R_5 \setminus R_1) \subset S_5$ and $(R \setminus R_1 \setminus R_5) \subset S_4$.

Thus $S_1 \cup S_2 \cup S_3 \cup S_4 \cup S_5 = V_C$. □

Corollary. The stage classification assigns each node to exactly one stage, ensuring no duplication or omission of nodes and therefore later variables and constraints during decomposition.

2 Appendix: Variable-Count Estimation

This appendix documents how subproblem complexity is estimated for runtime allocation. The estimates are computed from input data tables before the *MILP* model is built, counting expected variables based on entity attributes and horizon settings. Table 14 shows which attributes trigger discrete or continuous variables:

Entity Type	**Discrete Variables**	**Continuous Variables**
Production	+1 if discrete/minimum lot size active +1 if fixed cost active	+1 per active period
Transport	+1 if discrete/minimum lot size active +1 if fixed cost active	+1 per active period
Customer Transport	+1 if discrete/minimum lot size active +1 if fixed cost active	+1 per active period
External Procurement	+1 if fixed cost active	+1 per active period
Location Product	None	+1 stock variable per plannable period
Customer Product	None	+2 per demand period
Resources	None	None (constraints only)

Table 14: Variable estimation rules by entity type

When multiple features require a binary indicator, they share the same variable and may not need a continuous variable.

3 Appendix: Solver Configuration

The following Gurobi parameters were configured for all optimization runs in this evaluation. Parameters not listed use Gurobi's default values.

Parameter	**Value**	**Description**
NodeLimit	1e+100	Maximum number of branch-and-bound nodes (effectively unlimited)
FeasibilityTol	1e-08	Primal feasibility tolerance
IntFeasTol	1e-08	Integer feasibility tolerance
OptimalityTol	1e-06	Dual feasibility tolerance
MIPGap	1e-06	Relative MIP optimality gap
NumericFocus	2	Higher numeric precision focus
IntegralityFocus	1	Prioritize integrality of solutions

Table 15: Non-default Gurobi solver parameters